

Programación

None

None

None

programación

1. Bienvenid@ al módulo de Programación	3
1.1 Bolsa de proyectos	3
2. Sprint 1. Introducción a la programación	7
2.1 Semana 1. El nacimiento de la aplicación: estructura, variables y tipos de datos	7
2.2 Semana 2. El motor matemático: operadores, expresiones y conversiones de tipo	14
2.3 Semana 3. La arquitectura del código - constantes, escapes y printf	28
2.4 ---	31
2.5 ---	34
2.6 ---	37
3. Unidad 2	40

1. Bienvenid@ al módulo de Programación

El equipo de estudiantes debe elegir uno de los proyectos que se muestran a continuación.

1.1 Bolsa de proyectos

1.1.1 Ciberseguridad

1. Simulador de phishing — juego interactivo de detección de mensajes fraudulentos; clasificación de mensajes por tipología (Email, SMS, Red Social) con pistas contextuales, gestión de usuarios/sesiones, estadísticas de aciertos e historial de partidas.
2. Bóveda de contraseñas — gestor de credenciales con evaluación de fortaleza mediante políticas de seguridad (Básica, Estricta, Corporativa); control de caducidad, filtrado de cuentas guardadas y persistencia del almacén seguro.

1.1.2 Python

1. Calculadora paso a paso — evaluador de expresiones matemáticas con desglose paso a paso; tokenización, precedencia y cálculo con operadores polimórficos, interfaz con visor del proceso de resolución e histórico de cálculos.
2. Generador de datos de prueba — herramienta de síntesis de datasets sintéticos a partir de plantillas; campos polimórficos (Texto, Número, Fecha, Email), generación de lotes de registros aleatorios y exportación a ficheros CSV.

1.1.3 Videojuegos y realidad virtual

1. Aventura conversacional — motor de narrativa interactiva ramificada con toma de decisiones; gestión de escenas, inventario del jugador, interacción con diferentes tipos de personajes (Aliado, Enemigo, Neutral) y guardado del estado de la partida.
2. Simulador de físicas 2D — entorno de simulación cinemática con detección de colisiones y rebotes; escenario visual con múltiples bolas simultáneas de comportamiento polimórfico (Normal, Pesada, Rebote especial) y exportación de estadísticas de la simulación.

1.1.4 Productos software en contenedores

1. Reparto de contenedores en nodos — planificador de asignación de carga en clúster según capacidad de CPU y memoria; gestión de nodos especializados (Alta Memoria, Alta CPU), interfaz gráfica con barras de ocupación y persistencia de configuraciones de reparto.
2. Catálogo de servicios + generador de config — asistente paso a paso (wizard) para la definición y parametrización de servicios (Web, BD, Worker); generación polimórfica de ficheros de configuración, catálogo de servicios e historial de despliegues.

1.1.5 Inteligencia artificial y big data

1. Motor de recomendación — sistema de recomendación basado en afinidad entre gustos y catálogos de ítems categorizados (Película, Música, Libro); cálculo de puntuaciones de coincidencia por etiquetas, gestión de perfiles y persistencia de valoraciones.
2. Analizador de reseñas — analizador de sentimiento léxico sobre comentarios; procesamiento de palabras clave mediante diccionarios específicos según el tipo de producto, estadísticas agregadas por producto y persistencia del catálogo de reseñas.

1.1.6 Recursos y servicios en la nube

1. Cotizador cloud — presupuestador interactivo de infraestructura en la nube; cálculo polimórfico de costes según el tipo de recurso (Cómputo, Almacenamiento, Red), visor para montar y comparar presupuestos y persistencia de ofertas para clientes.

2. Dashboard de monitorización — panel de control visual para métricas y telemetría de sistemas; simulación de lecturas de sensores (CPU, Memoria, Red), series temporales, detección de umbrales con disparo de alertas e histórico persistido de incidencias.

# Proyecto (especialización)	S1 · RA1	S2 · RA3	S3 · RA2	S4 · RA4	S5 · RA6
1. Simulador de phishing (Ciberseguridad)	Clasificar 1 mensaje fijo sí/no	Menú que recorre varios mensajes, cuenta aciertos	<code>Random</code> para elegir mensaje al azar; <code>String</code> (<code>contains</code> , <code>toLowerCase</code>) para buscar palabras sospechosas; <code>Scanner</code> para la respuesta del usuario	Clases <code>Mensaje</code> , <code>Usuario</code> , <code>Sesión</code>	Lista de mensajes + estadísticas de aciertos
2. Bóveda de contraseñas (Ciberseguridad)	Evaluar longitud de 1 contraseña	Reglas de fortaleza (mayúsc./dígitos/símbolos)	<code>String</code> (<code>length</code> , <code>matches</code> con <code>regex</code>) para evaluar la contraseña; <code>LocalDate</code> para fecha de caducidad	Clases <code>Credencial</code> , <code>PoliticaSeguridad</code>	Lista de credenciales guardadas, filtrado
3. Calculadora paso a paso (Python)	Evaluar suma/resta simple	Precedencia de operadores, tokenizar con bucles	<code>StringBuilder</code> para construir la expresión; clases envoltorio <code>Integer</code> / <code>Double</code> (<code>parseInt</code> , <code>parseDouble</code>); <code>Math</code> (métodos estáticos) para operaciones	Clases <code>Expresión</code> , <code>Token</code> , <code>Operador</code>	Lista de pasos de resolución (histórico)
4. Generador de datos de prueba (Python)	Generar 1 dato aleatorio de lista fija	Bucle para generar N datos	<code>Random</code> para valores aleatorios; <code>LocalDate</code> para fechas; <code>String.format</code> para dar formato al dato generado	Clases <code>Plantilla</code> , <code>Campo</code> , <code>Registro</code>	Colección de plantillas y lotes generados
5. Aventura conversacional (Videojuegos)	Texto de intro + 1 decisión	Ramificación de la historia con control de flujo	<code>Scanner</code> para las decisiones del jugador; <code>Random</code> para eventos aleatorios; <code>String</code> para comparar respuestas	Clases <code>Personaje</code> , <code>Escena</code> , <code>Objeto</code>	Inventario del jugador, lista de escenas
6. Simulador de físicas 2D (VR/Videojuegos)	Mover 1 bola con variables	Bucle de simulación con	<code>Math</code> (<code>sqrt</code> , <code>pow</code>) para cálculos de	Clases <code>Bola</code> , <code>Escenario</code>	Colección de bolas simultáneas

# Proyecto (especialización)	S1 · RA1	S2 · RA3	S3 · RA2	S4 · RA4	S5 · RA6
	posición/ velocidad	colisión en bordes	velocidad/ colisión; Random para posición/ velocidad inicial		
7. Reparto de contenedores en nodos (Contenedores)	Comprobar si 1 contenedor cabe en 1 nodo	Bucle de reparto con reglas simples	Math para comprobar capacidad restante; Random para generar contenedores de prueba	Clases Contenedor, Nodo, Cluster	Listas de contenedores/ nodos, algoritmo de reparto
8. Catálogo de servicios + generador de config (Contenedores)	Pedir datos de 1 servicio	Asistente (wizard) paso a paso	StringBuilder para construir el fichero de configuración; LocalDateTime para la marca de tiempo de la config	Clases Servicio, Configuración	Catálogo de servicios + historial de configs
9. Motor de recomendación (IA/Big Data)	Mostrar 1 recomendación fija	Bucle que compara gustos con catálogo pequeño	Random para simular gustos iniciales; Math para calcular puntuación de coincidencia	Clases Item, Usuario, Etiqueta	Catálogo + gustos + cálculo de coincidencias
10. Analizador de reseñas (IA/ Big Data)	Contar palabras +/- en 1 texto	Bucle que analiza varias reseñas	String (split, contains, toLowerCase) para contar palabras clave en la reseña	Clases Producto, Reseña, Autor	Reseñas por producto + estadísticas
11. Cotizador cloud (Nube)	Coste de 1 recurso (fórmula fija)	Elegir tipo de recurso y aplicar su fórmula	Math y clases envoltorio (Double) para cálculo de coste; String.format para mostrar el importe	Clases Recurso, Presupuesto	Recursos de un presupuesto + lista de presupuestos
12. Dashboard de monitorización (Nube)	Mostrar 1 valor aleatorio de CPU	Serie temporal + detección de umbral	Random para simular lecturas de sensor; LocalDateTime para la marca temporal de cada medición	Clases Sensor, Métrica, Alerta	Histórico de métricas + lista de alertas

2. Sprint 1. Introducción a la programación

2.1 Semana 1. El nacimiento de la aplicación: estructura, variables y tipos de datos

A lo largo de este sprint desarrollaremos y evolucionaremos **un único archivo de pseudocódigo (`ControlAccesoQR.psc`) y una única clase Java (`ControlAccesoQR.java`)** para el caso guía del **IES El Caminàs**, mientras cada estudiante hace evolucionar de forma paralela el archivo único de **su proyecto elegido de la bolsa de proyectos**.

2.1.1 Día 1 - 2 sesiones

1. Caso guía en AzaharTech

Son las tres de la tarde en la sede de **AzaharTech** en Castellón de la Plana. **Laia Claramunt** conecta su portátil al proyector principal. En pantalla aparece el entorno de desarrollo IntelliJ IDEA completamente vacío:

«Equipo, comenzamos el desarrollo del sistema de acceso para el IES El Caminàs; hoy abrimos el archivo oficial de la aplicación: `ControlAccesoQR`.

Una aplicación profesional crece capa a capa. El equipo directivo del instituto necesita que el terminal empiece registrando los parámetros físicos del vestíbulo: el identificador del terminal que controla la pantalla y la temperatura ambiente del sensor térmico.

Hoy crearemos el esqueleto del programa, aprenderemos qué ocurre en la memoria RAM al reservar variables numéricas (`int` y `double`) y capturaremos los primeros datos desde el teclado».

2. Fundamento teórico: estructura del programa y variables numéricas en RAM

EL MODELO DE MEMORIA DEL TERMINAL			
Variable	Tipo en Java	Espacio en RAM	Valor almacenado
<code>terminalId</code>	<code>int</code>	32 bits (4 bytes)	101 (Número entero)
<code>tempVestibulo</code>	<code>double</code>	64 bits (8 bytes)	21.5 (Coma flotante)

- La clase como contenedor maestro.** En Java, todo código pertenece a una clase (`public class ControlAccesoQR`). El archivo en disco debe llamarse exactamente igual: `ControlAccesoQR.java`.
- El método de entrada (`main`).** La JVM busca la instrucción `public static void main(String[] args)` para iniciar la ejecución secuencial de arriba hacia abajo.
- Variables numéricas primitivas.**
 - `int` :** Almacena números enteros sin decimales (de `-$2.147$` a `+$2.147$` millones).
 - `double` :** Almacena números reales con precisión decimal de 64 bits.
- Captura con `Scanner` .** Creamos un canal de lectura interactivo (`Scanner teclado = new Scanner(System.in);`) que captura enteros con `nextInt()` y decimales con `nextDouble()`.

3. Algorítmica y traducción a Java de la app `ControlAccesoQR v0.1`

PASO A. ALGORITMO EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc — V0.1`)

```

Algoritmo ControlAccesoQR
  Definir terminalId Como Entero
  Definir tempVestibulo Como Real

  Escribir "ID del terminal:"
  Leer terminalId
  Escribir "Temperatura del sensor (°C):"
  Leer tempVestibulo

  Escribir "Terminal configurado: #", terminalId
  Escribir "Lectura térmica:      ", tempVestibulo, " °C"
FinAlgoritmo

```

PASO B. TRADUCCIÓN A JAVA (pr/src/ControlAccesoQR.java — V0.1)

```

import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (°C): ");
        tempVestibulo = teclado.nextDouble();

        System.out.println("Terminal configurado: #" + terminalId);
        System.out.println("Lectura térmica:      " + tempVestibulo + " °C");

        teclado.close();
    }
}

```

4. Trabajo del estudiante: nacimiento de su proyecto propio

Cada estudiante crea en su carpeta `pr/pseudocodigo/` el archivo maestro de su proyecto propio y en `pr/src/` la clase `.java v0.1`:

- Declara dos variables numéricas (`int` y `double`) a su proyecto:
- En Aventura conversacional: puntos de vida (`int puntosVida`) y multiplicador de daño (`double factorDano`).
- En Motor de recomendación: identificador de ítem (`int idItem`) y valoración media (`double valoracionMedia`).
- En Simulador de físicas 2D: número de colisiones (`int totalColisiones`) y masa del cuerpo (`double masaKg`).
- En Bóveda de contraseñas: días para la caducidad (`int diasCaducidad`) y porcentaje de fortaleza (`double nivelSeguridad`).»
- Compila y verifica la ejecución en la consola de IntelliJ.

2.1.2 Día 2 - 2 sesiones

1. Caso guía en AzaharTech

Es martes por la tarde. **Pau Ferrer** ejecuta `ControlAccesoQR v0.1` y muestra la pantalla:

«El terminal ya arranca y guarda el número de terminal `101` y la temperatura `21.5`. Pero cuando una persona acerca el móvil a la pantalla del vestíbulo, el sistema no sabe a quién pertenece ese escaneo».

Alba Torres toma el teclado y abre el archivo de ayer:

«No vamos a crear un programa nuevo. Vamos a evolucionar `ControlAccesoQR.java`. Añadiremos los campos para el nombre de la persona, su DNI, el tipo de relación con el instituto (estudiante, docente, visita) y bandera lógica que indique si está entrando o saliendo.

Pero atención a la trampa de Java: al leer números antes que textos, el buffer del teclado guarda un salto de línea invisible (`\n`) que debemos limpiar para que el programa no se salte la lectura del nombre».

2. Fundamento teórico: caracteres, cadenas y el buffer del Scanner

EL PROBLEMA DEL BUFFER RESIDUAL EN SCANNER		
Usuario teclea:	[1][0][1][\n (Enter)]	
nextInt() lee:	[1][0][1]	→ Deja el [\n] flotando en el buffer.
nextLine() lee:	[\n]	→ Cree que el usuario pulsó Enter vacío.
SOLUCIÓN:	teclado.nextLine();	→ Limpia el buffer antes del texto real.

1. char frente a String:

- `char letraGrupo = 'B';`: Un único carácter delimitado obligatoriamente por comillas simples (`' '`).
- `String nombre = "Juan Pérez";`: Objeto que gestiona texto de cualquier longitud entre comillas dobles (`" "`).

2. boolean (Estado del sistema):

Almacena `true` o `false`. Ideal para variables de control como `matriculaActiva`.

3. Limpieza del buffer:

Siempre que se invoque `nextInt()` o `nextDouble()` y la siguiente instrucción sea `nextLine()`, se debe intercalar una llamada a `teclado.nextLine();` para vaciar el salto de línea residual.

3. Evolución a ControlAccesoQR v0.2

Observa cómo el código de ayer se amplía añadiendo los bloques de identidad sin eliminar lo anterior.

PASO A. EVOLUCIÓN EN PSEINT (pr/pseudocodigo/ControlAccesoQR.psc — V0.2)

```

Algoritmo ControlAccesoQR
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real

    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (°C):"
    Leer tempVestibulo

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Terminal configurado: #", terminalId
    Escribir "Lectura térmica: ", tempVestibulo, " °C"
    Escribir "Persona: ", nombrePersona, " (DNI: ", dniPersona, ")"
    Escribir "Perfil: ", perfilPersona
    Escribir "Sentido del paso: Entrada (", esEntrada, ")"

FinAlgoritmo

```

PASO B. EVOLUCIÓN EN JAVA (pr/src/ControlAccesoQR.java — V0.2)

Abrimos el archivo `ControlAccesoQR.java` en IntelliJ y lo modificamos directamente:

```

import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;

        String dniPersona;
        String nombrePersona;
        char perfilPersona;
        boolean esEntrada;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (°C): ");
        tempVestibulo = teclado.nextDouble();
        teclado.nextLine();
    }
}

```

```

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita):");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.println("Terminal configurado: #" + terminalId);
        System.out.println("Lectura térmica: " + tempVestibulo + " °C");
        System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
        System.out.println("Perfil: " + perfilPersona);
        System.out.println("Sentido del paso:  Entrada (" + esEntrada + ")");

        teclado.close();
    }
}

```

4. Trabajo del estudiante: evolución de su proyecto propio

Cada estudiante abre sus archivos `.psc` y `.java`:

- Añade los campos alfanuméricos de su proyecto:
- En Aventura conversacional: nombre del héroe (String), clase de personaje 'G' de Guerrero, 'M' de Mago, 'P' de Pícaro (char) y si la partida está activa (boolean estaVivo).
- En Motor de recomendación: título de la película/libro (String), tipo de contenido 'P' de Película, 'M' de Música, 'L' de Libro (char) y si está marcado como favorito (boolean esFavorito).
- En Simulador de físicas 2D: etiqueta del cuerpo (String), tipo de partícula 'N' de Normal, 'P' de Pesada, 'R' de Rebote especial (char) y si tiene la gravedad activada (boolean gravedadActiva).
- En Bóveda de contraseñas: nombre del servicio o web (String), nivel de política 'B' de Básica, 'E' de Estricta, 'C' de Corporativa (char) y si requiere doble factor (boolean requiere2FA).»
- Aplica la limpieza del buffer con `teclado.nextLine()`.
- Ejecuta pruebas verificando que se pueden introducir nombres con espacios sin saltos inesperados.

2.1.3 Día 3 - 2 sesiones

1. Caso guía en AzaharTech

Es miércoles por la tarde. **Laia Claramunt** revisa la versión v0.2:

«El sistema ya sabe qué terminal lee, la temperatura del vestíbulo y quién pasa. Ahora el IES El Caminàs nos pide procesar el tiempo que la persona permanece en el instituto: para cada acceso debemos calcular los minutos que hay entre la hora de entrada y la hora de salida.

Hoy aprenderemos a operar matemáticamente sobre las variables de nuestro programa y a componer un mensaje unificado donde convivan números y texto sin que el operador `+` distorsione los cálculos».

2. Fundamento teórico: aritmética y concatenación segura

1. La sobrecarga del operador `+`:

- Si ambos operandos son numéricos: realiza una **suma matemática** (`50 + 50 = 100`).
- Si al menos un operando es una cadena de texto: realiza una **concatenación** (unión de textos).

2. Evaluación de izquierda a derecha:

```
System.out.println("Minutos: " + 50 + 50); // Imprime "Minutos: 5050" (¡ERROR!)
System.out.println("Minutos: " + (50 + 50)); // Imprime "Minutos: 100" (CORRECTO)
```

3. Multiplicación y prioridad: La multiplicación (`*`) se evalúa antes que la suma (`+`), a menos que usemos paréntesis (`()`).

3. Evolución a ControlAccesoQR v0.3

Ampliamos el código de la versión v0.2 añadiendo el bloque de cálculo de sesiones lectivas y la composición del mensaje de confirmación.

PASO A. EVOLUCIÓN EN PSEINT ([pr/pseudocodigo/ControlAccesoQR.psc](#) — V0.3)

```
Algoritmo ControlAccesoQR
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real

    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada Como Entero
    Definir minutosTotalesSalida Como Entero
    Definir minutosEstanciaTotal Como Entero
    Definir tokenResumen Como Cadena

    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (°C):"
    Leer tempVestibulo

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada

    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * 60) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * 60) + minutoSalida
    minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

    tokenResumen <- dniPersona + "-ESTANCIA-" + ConvertirATexto(minutosEstanciaTotal)

    Escribir "Terminal configurado: #", terminalId
    Escribir "Lectura térmica: ", tempVestibulo, " °C"
    Escribir "Persona: ", nombrePersona, " (DNI: ", dniPersona, ")"
    Escribir "Perfil: ", perfilPersona
    Escribir "Sentido del paso: Entrada (", esEntrada, ")"
    Escribir "Token: ", tokenResumen
    Escribir "Horario: Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
    Escribir "Permanencia total en centro: ", minutosEstanciaTotal, " minutos."
FinAlgoritmo
```

PASO B. EVOLUCIÓN EN JAVA ([pr/src/ControlAccesoQR.java](#) — V0.3)

Abrimos nuestro archivo `ControlAccesoQR.java` y lo evolucionamos:

```
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;

        String dniPersona;
        String nombrePersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada;
```

```

int minutoEntrada;
int horaSalida;
int minutoSalida;
int minutosTotalesEntrada;
int minutosTotalesSalida;
int minutosEstanciaTotal;
String tokenResumen;

System.out.print("ID del terminal: ");
terminalId = teclado.nextInt();

System.out.print("Temperatura del sensor (°C): ");
tempVestibulo = teclado.nextDouble();
teclado.nextLine();

System.out.print("DNI de la persona: ");
dniPersona = teclado.nextLine();

System.out.print("Nombre completo: ");
nombrePersona = teclado.nextLine();

System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita):");
perfilPersona = teclado.next().charAt(0);

esEntrada = true;

System.out.print("Hora de entrada (0-23): ");
horaEntrada = teclado.nextInt();

System.out.print("Minuto de entrada (0-59): ");
minutoEntrada = teclado.nextInt();

System.out.print("Hora de salida (0-23): ");
horaSalida = teclado.nextInt();

System.out.print("Minuto de salida (0-59): ");
minutoSalida = teclado.nextInt();

minutosTotalesEntrada = (horaEntrada * 60) + minutoEntrada;
minutosTotalesSalida = (horaSalida * 60) + minutoSalida;
minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

tokenResumen = dniPersona + "-ESTANCIA-" + minutosEstanciaTotal;

System.out.println("Terminal configurado: #" + terminalId);
System.out.println("Lectura térmica: " + tempVestibulo + " °C");
System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
System.out.println("Perfil: " + perfilPersona);
System.out.println("Sentido del paso: Entrada (" + esEntrada + ")");
System.out.println("Token: " + tokenResumen);
System.out.println("Horario: Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
System.out.println("Permanencia total en centro: " + minutosEstanciaTotal + " minutos.");

teclado.close();
}
}

```

4. Trabajo del estudiante: evolución de su proyecto propio

El estudiante abre su archivo `.java`:

- Incorpora el cálculo aritmético secuencial adaptado a su proyecto.
- En Aventura conversacional: calcular el daño total ($\text{danoBase} * \text{factorDano}$) y restar la vida restante ($\text{puntosVida} - \text{danoTotal}$).
- En Motor de recomendación: sumar puntuaciones por afinidad ($\text{afinidadGenero} + \text{afinidadDirector}$) y calcular la diferencia respecto a la media de la comunidad ($\text{totalAfinidad} - \text{mediaComunidad}$).
- En Simulador de físicas 2D: calcular el desplazamiento lineal restando la posición inicial a la posición final calculada ($\text{posFinal} - \text{posInicial}$).
- En Bóveda de contraseñas: calcular los días de vigencia restantes restando los días transcurridos al límite de caducidad ($\text{diasCaducidad} - \text{diasTranscurridos}$).»
- Utiliza paréntesis `()` para proteger una operación dentro de los mensajes de salida.
- Compila y verifica que la concatenación no genera errores numéricos.

2.1.4 Día 4 - 2 sesiones

1. Caso guía en AzaharTech

Es jueves por la tarde. Concluyen las primeras 8 horas de Programación. **Laia Claramunt** revisa la versión v0.3 en el repositorio:

«Fijaos en lo que habéis logrado en solo cuatro días: tenemos una aplicación viva que ya sabe capturar datos del hardware, identificar al usuario y calcular tiempos de estancia en el centro educativo.

Hoy cada uno de vosotros va a dedicar estas dos horas a auditar, limpiar y documentar la versión v0.3 de su proyecto propio de la bolsa de proyectos. Aplicaremos la indentación oficial, cerraremos recursos y realizaremos el commit formal en GitHub».

2. Estándares de calidad de código en AzaharTech

1. **Autoformato en IntelliJ.** Todo código debe pasar por el formateador automático del IDE pulsando **Ctrl + Alt + L** (en GNU/Linux). La indentación debe ser homogénea de 4 espacios.
2. **Nombres descriptivos.** Las variables deben reflejar su propósito (`minutosTotalesLectivos`, no `mtl`).
3. **Cierre de recursos.** Toda clase que utilice `Scanner` debe cerrarlo explícitamente con `.close()` al final del `main`.

3. La versión v0.3 del proyecto propio del estudiante

Cada estudiante verifica que su archivo único `pr/src/MiProyecto.java` cumple con el nivel de consolidación alcanzado en el caso guía:

```
import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;

        String dniPersona;
        String nombrePersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada;
        int minutoEntrada;
        int horaSalida;
        int minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
        int minutosEstanciaTotal;
        String tokenResumen;

        System.out.println("=====");
        System.out.println("  AZAHARTECH - TERMINAL DE ACCESO VESTÍBULO  ");
        System.out.println("    Cliente: IES El Caminàs (Curso 2026/2027)    ");
        System.out.println("=====");
        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (°C): ");
        tempVestibulo = teclado.nextDouble();
        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();
    }
}
```

```

System.out.print("Minuto de salida (0-59): ");
minutoSalida = teclado.nextInt();

minutosTotalesEntrada = (horaEntrada * 60) + minutoEntrada;
minutosTotalesSalida = (horaSalida * 60) + minutoSalida;

minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

tokenResumen = dniPersona + "-ESTANCIA-" + minutosEstanciaTotal;

System.out.println("-----");
System.out.println("Terminal configurado:      #" + terminalId);
System.out.println("Lectura térmica:           " + tempVestibulo + " °C");
System.out.println("Persona:                   " + nombrePersona + " (DNI: " + dniPersona + ")");
System.out.println("Perfil:                     " + perfilPersona);
System.out.println("Sentido del paso:          Entrada (" + esEntrada + ")");
System.out.println("Token:                     " + tokenResumen);
System.out.println("Horario:                   Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
System.out.println("Permanencia total en centro: " + minutosEstanciaTotal + " minutos.");

teclado.close();
}
}

```

4. (Hacerlo únicamente si el grupo de estudiantes ya ha sincronizado con GitHub en ED) Cierre formal en git y sincronización con GitHub

El estudiante confirma los avances de la primera semana en su repositorio:

1. Pulsa el atajo **ctrl + K** (o haz clic en el icono verde de verificación **Commit** en la barra lateral izquierda).
2. En el panel de Commit, marca las casillas de los archivos modificados dentro de `pr/` (`MiProyecto.java` y `MiProyecto.psc`).
3. En la caja de texto para el mensaje, escribe siguiendo el estándar convencional:
`feat(pr): consolidar version v0.3 con captura de datos, calculos aritmeticos y limpieza de codigo`
4. Despliega el botón azul inferior y selecciona **«Commit and Push»**.
5. En la ventana de confirmación que aparece, pulsa **Push** para enviar los cambios a GitHub.

2.2 Semana 2. El motor matemático: operadores, expresiones y conversiones de tipo

Continuamos trabajando sobre el **mismo archivo maestro** que dejamos al final de la Semana 1. Tomamos la versión `ControlAccesoQR v0.3` y la evolucionamos progresivamente hasta la versión `v0.6` integrando operadores compuestos, descomposición con módulo (%) y conversiones de tipo (*casting*), mientras cada estudiante replica esta misma evolución en la clase única de **su proyecto elegido de la bolsa de proyectos**.

2.2.1 Día 5 - 2 sesiones

1. Caso guía en AzaharTech

Es lunes por la tarde. En la sala técnica de **AzaharTech**, **Pau Ferrer** abre el archivo `ControlAccesoQR.java` tal y como quedó el jueves anterior (versión v0.3). Quiere añadir un contador para saber cuántos fichajes procesa el terminal a lo largo de la mañana y ha escrito:

```

personasEnCentro = personasEnCentro + 1;
aforoDisponible = aforoDisponible - 1;
idUltimoFichaje = idUltimoFichaje + 1;
tokenResumen = tokenResumen + "-REG" + idUltimoFichaje;

```

Alba Torres se acerca a su monitor, señala la pantalla y le explica:

«Pau, en un código profesional no duplicamos el nombre de la variable a ambos lados del signo igual. Para acumular valores o contar eventos utilizamos operadores de asignación compuesta (`+=`) y el operador de incremento (`++`).

Hacen que el código sea más compacto, reducen el riesgo de erratas al escribir nombres largos y permiten al compilador generar un código intermedio más optimizado.

Hoy abriremos nuestro archivo `ControlAccesoQR` y lo refactorizaremos a la versión v0.4: sustituiremos las asignaciones largas por operadores compuestos y aprenderemos a distinguir el pre-incremento del post-incremento para evitar efectos secundarios en la memoria».

2. Fundamento teórico: asignación compuesta, incremento y mutabilidad de memoria

OPERADORES DE ASIGNACIÓN COMPUESTA EN JAVA		
Expresión compacta	Expresión equivalente	Acción sobre la celda de memoria RAM
<code>total += valor;</code>	<code>total = total + valor;</code>	Suma 'valor' al dato actual de total
<code>texto += " nuevo";</code>	<code>texto = texto + valor;</code>	Concatena 'texto' al final de cadena
<code>tiempo -= pausa;</code>	<code>tiempo = tiempo - pausa;</code>	Resta 'pausa' al dato actual
<code>aforo *= factor;</code>	<code>aforo = aforo * factor;</code>	Multiplica el contenido por 'factor'
<code>cupos /= divisor;</code>	<code>cupos = cupos / divisor;</code>	Divide el contenido entre 'divisor'

A. OPERADORES UNARIOS DE INCREMENTO (`++`) Y DECREMENTO (`--`)

Aumentan o disminuyen el valor de una variable entera exactamente en una unidad:

- **Post-incremento (`variable++`).** El valor actual de la variable se utiliza en la expresión donde se encuentra y, **justo después de ser leído**, la variable se incrementa en 1 en la memoria RAM.
- **Pre-incremento (`++variable`).** La variable se incrementa en 1 en la memoria RAM **antes** de que su valor sea leído o utilizado en la expresión circundante.

```
// Ejemplo de análisis de memoria en AzaharTech:
int accesos = 10;
System.out.println(accesos++); // Imprime 10 en consola. En memoria RAM pasa a valer 11.
System.out.println(accesos);   // Imprime 11.
System.out.println(++accesos); // En memoria RAM sube a 12 y luego imprime 12.
```

3. Refactorización a `ControlAccesoQR v0.4`

Tomamos el archivo único de la Semana 1 y lo refactorizamos incorporando el contador global del terminal y asignaciones compuestas.

PASO A. ACTUALIZACIÓN EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V0.4)

```
Algoritmo ControlAccesoQR
  Definir terminalId Como Entero
  Definir tempVestibulo Como Real
  Definir nombrePersona, dniPersona Como Cadena
  Definir perfilPersona Como Caracter
  Definir esEntrada Como Logico

  Definir horaEntrada, minutoEntrada Como Entero
  Definir horaSalida, minutoSalida Como Entero
  Definir minutosTotalesEntrada Como Entero
  Definir minutosTotalesSalida Como Entero
  Definir minutosEstanciaTotal Como Entero
  Definir tokenResumen Como Cadena

  Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
  personasEnCentro <- 120
  aforoDisponible <- 80
  idUltimoFichaje <- 1042

  Escribir "ID del terminal:"
```

```

Leer terminalId
Escribir "Temperatura del sensor (°C):"
Leer tempVestibulo

Escribir "DNI de la persona:"
Leer dniPersona
Escribir "Nombre completo:"
Leer nombrePersona
Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
Leer perfilPersona

esEntrada <- Verdadero

Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
Leer horaEntrada
Leer minutoEntrada
Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
Leer horaSalida
Leer minutoSalida

minutosTotalesEntrada <- (horaEntrada * 60) + minutoEntrada
minutosTotalesSalida <- (horaSalida * 60) + minutoSalida
minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

personasEnCentro <- personasEnCentro + 1
aforoDisponible <- aforoDisponible - 1
idUltimoFichaje <- idUltimoFichaje + 1

tokenResumen <- dniPersona
tokenResumen <- tokenResumen + "-T" + ConvertirATexto(terminalId)
tokenResumen <- tokenResumen + "-ESTANCIA-" + ConvertirATexto(minutosEstanciaTotal)
tokenResumen <- tokenResumen + "-REG" + ConvertirATexto(idUltimoFichaje)

Escribir "Terminal configurado: #", terminalId
Escribir "Lectura térmica: ", tempVestibulo, " °C"
Escribir "Persona: ", nombrePersona, " (DNI: ", dniPersona, ")"
Escribir "Perfil: ", perfilPersona
Escribir "Sentido del paso: Entrada (", esEntrada, ")"
Escribir "Token: ", tokenResumen
Escribir "Horario: Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
Escribir "Permanencia total en centro: ", minutosEstanciaTotal, " minutos."
Escribir "Estado actual del aforo:"
Escribir "- personas en el centro: ", personasEnCentro, " (+1)"
Escribir "- plazas libres: ", aforoDisponible, " (-1)"
FinAlgoritmo

```

PASO B. REFACTORIZACIÓN EN JAVA (pr/src/ControlAccesoQR.java — V0.4)

Abrimos nuestro archivo maestro `ControlAccesoQR.java` y aplicamos directamente la refactorización:

```

import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;
        String nombrePersona, dniPersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
        int minutosEstanciaTotal;
        String tokenResumen;

        int personasEnCentro = 120;
        int aforoDisponible = 80;
        int idUltimoFichaje = 1042;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (°C): ");
        tempVestibulo = teclado.nextDouble();

        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;
    }
}

```

```

        System.out.print("Hora de entrada (0-23): ");
        horaEntrada = teclado.nextInt();

        System.out.print("Minuto de entrada (0-59): ");
        minutoEntrada = teclado.nextInt();

        System.out.print("Hora de salida (0-23): ");
        horaSalida = teclado.nextInt();

        System.out.print("Minuto de salida (0-59): ");
        minutoSalida = teclado.nextInt();

        minutosTotalesEntrada = (horaEntrada * 60) + minutoEntrada;
        minutosTotalesSalida = (horaSalida * 60) + minutoSalida;
        minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

        personasEnCentro++;
        aforoDisponible--;
        idUltimoFichaje++;

        tokenResumen = dniPersona;
        tokenResumen += "-T" + terminalId;
        tokenResumen += "-ESTANCIA-" + minutosEstanciaTotal;
        tokenResumen += "-REG" + idUltimoFichaje;

        System.out.println("Terminal configurado: #" + terminalId);
        System.out.println("Lectura térmica: " + tempVestibulo + " °C");
        System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
        System.out.println("Perfil: " + perfilPersona);
        System.out.println("Sentido del paso: Entrada (" + esEntrada + ")");
        System.out.println("Token: " + tokenResumen);
        System.out.println("Horario: Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
        System.out.println("Permanencia total en centro: " + minutosEstanciaTotal + " minutos.");
        System.out.println("Estado actual del aforo:");
        System.out.println("- personas en el centro: " + personasEnCentro + " (+1)");
        System.out.println("- plazas libres: " + aforoDisponible + " (-1)");

        teclado.close();
    }
}

```

4. Trabajo del estudiante: refactorización de su proyecto propio

El estudiante abre su archivo `.java` (que dejó en v0.3 el jueves anterior):

- Aplica los operadores unarios de incremento (`++`) y decremento (`--`) sobre contadores de eventos o estados numéricos de su sistema:
- En **Aventura conversacional**: incrementar el contador de turnos de la escena y decrementar los puntos de energía o pociones disponibles.
- En **Motor de recomendación**: incrementar el contador de recomendaciones emitidas y decrementar las consultas gratuitas restantes del perfil.
- En **Simulador de físicas 2D**: incrementar el contador de impactos o colisiones detectadas y decrementar las partículas activas en pantalla.
- En **Bóveda de contraseñas**: incrementar el identificador correlativo de la nueva credencial y decrementar los intentos de autenticación permitidos.
- Utiliza el operador de asignación compuesta (`+=`) para construir paso a paso la cadena de texto identificativa (token, resumen o registro de auditoría):
- En **Aventura conversacional**: ir componiendo el registro de la partida sobre la misma variable.
- En **Motor de recomendación**: construir el código de afinidad concatenando partes.
- En **Simulador de físicas 2D**: acumular los datos en la traza de telemetría.
- En **Bóveda de contraseñas**: ensamblar el registro de auditoría de la credencial.
- Compila y ejecuta en IntelliJ comprobando que los contadores actualizan su valor en memoria y que la cadena compuesta con `+=` se muestra completa en la consola.

2.2.2 Día 6 - 2 sesiones

1. Caso guía en AzaharTech

Es martes por la tarde. En la pantalla de control, **Pau Ferrer** muestra que el reloj interno del vestíbulo del IES El Caminàs devuelve el tiempo de actividad del terminal en **segundos brutos acumulados**: 7540 segundos.

Pau comenta:

«En la pantalla no podemos mostrarle al conserje que el terminal lleva '7540 segundos encendido'. El cliente necesita ver cuántas horas completas, minutos sobrantes y segundos exactos representa ese número.

He intentado dividir `7540 / 60`, pero me da 125 minutos y no sé cómo extraer las horas y los segundos sin escribir un algoritmo larguísimo con restas».

Alba Torres toma el mando en la pizarra:

«No necesitas restas ni condicionales. La combinación de la división entera (`/`) y el operador módulo o resto (`%`) resuelve este problema en tres líneas de código puramente secuenciales.

Hoy abriremos `ControlAccesoQR.java` y lo evolucionaremos a la versión v0.5: convertiremos nuestro programa en un motor capaz de descomponer cualquier magnitud temporal de forma exacta».

2. La división entera frente al operador residuo (%)

DESCOMPOSICIÓN MATEMÁTICA DE MAGNITUDES	
Total: 7540 segundos	
1. Horas completas:	$7540 / 3600 = 2$ horas (División entera descarta decimales)
2. Segundos restantes:	$7540 \% 3600 = 340$ segundos sobrantes que no llegan a 1 hora
3. Minutos de ese resto:	$340 / 60 = 5$ minutos
4. Segundos finales:	$7540 \% 60 = 40$ segundos restantes
RESULTADO EXACTO:	2 horas, 5 minutos y 40 segundos.

1. La división entre enteros en Java (`int / int`):

- Realiza un truncamiento automático hacia cero, descartando cualquier parte fraccionaria.
- `7540 / 3600` da exactamente `2`.

2. El operador módulo (%):

- Devuelve el residuo que no ha podido ser absorbido por la división entera.
- `7540 % 60` da exactamente `40` (porque $60 \times 125 = 7500$; sobran 40).

3. Poder algorítmico secuencial:

Permite desglosar monedas, tiempos, paquetes o ciclos de turnos sin requerir ninguna estructura condicional `if`.

3. Evolución a `ControlAccesoQR v0.5`

Ampliamos el archivo maestro `ControlAccesoQR` incorporando la lectura de segundos de actividad del terminal y su descomposición horaria.

PASO A. EVOLUCIÓN EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V0.5)

```
Algoritmo ControlAccesoQR
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real
    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada Como Entero
    Definir minutosTotalesSalida Como Entero
    Definir minutosEstanciaTotal Como Entero
    Definir tokenResumen Como Cadena
```

```

Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
personasEnCentro <- 120
aforoDisponible <- 80
idUltimoFichaje <- 1042

Definir segundosActividadTerminal Como Entero
Definir horasUptime, minutosUptime, segundosUptime Como Entero

Escribir "ID del terminal:"
Leer terminalId
Escribir "Temperatura del sensor (°C):"
Leer tempVestibulo
Escribir "Segundos de actividad del terminal (uptime):"
Leer segundosActividadTerminal

Escribir "DNI de la persona:"
Leer dniPersona
Escribir "Nombre completo:"
Leer nombrePersona
Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
Leer perfilPersona

esEntrada <- Verdadero

Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
Leer horaEntrada
Leer minutoEntrada
Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
Leer horaSalida
Leer minutoSalida

minutosTotalesEntrada <- (horaEntrada * 60) + minutoEntrada
minutosTotalesSalida <- (horaSalida * 60) + minutoSalida
minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

personasEnCentro <- personasEnCentro + 1
aforoDisponible <- aforoDisponible - 1
idUltimoFichaje <- idUltimoFichaje + 1

tokenResumen <- dniPersona
tokenResumen <- tokenResumen + "-T" + ConvertirATexto(terminalId)
tokenResumen <- tokenResumen + "-ESTANCIA-" + ConvertirATexto(minutosEstanciaTotal)
tokenResumen <- tokenResumen + "-REG" + ConvertirATexto(idUltimoFichaje)

horasUptime <- trunc(segundosActividadTerminal / 3600)
minutosUptime <- trunc((segundosActividadTerminal MOD 3600) / 60)
segundosUptime <- segundosActividadTerminal MOD 60

Escribir "Terminal configurado: #", terminalId
Escribir "Lectura térmica: ", tempVestibulo, " °C"
Escribir "Persona: ", nombrePersona, " (DNI: ", dniPersona, ")"
Escribir "Perfil: ", perfilPersona
Escribir "Sentido del paso: Entrada (", esEntrada, ")"
Escribir "Token: ", tokenResumen
Escribir "Horario: Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
Escribir "Permanencia total en centro: ", minutosEstanciaTotal, " minutos."
Escribir "Estado actual del aforo:"
Escribir "- personas en el centro: ", personasEnCentro, " (+1)"
Escribir "- plazas libres: ", aforoDisponible, " (-1)"
Escribir horasUptime, " horas, ", minutosUptime, " minutos y ", segundosUptime, " segundos en servicio."
FinAlgoritmo

```

PASO B. EVOLUCIÓN EN JAVA (pr/src/ControlAccesoQR.java — V0.5)

Abrimos nuestro archivo `ControlAccesoQR.java` y lo evolucionamos a v0.5:

```

import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;
        String nombrePersona, dniPersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
        int minutosEstanciaTotal;
        String tokenResumen;

        int personasEnCentro = 120;
        int aforoDisponible = 80;
        int idUltimoFichaje = 1042;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;
    }
}

```

```

System.out.print("ID del terminal: ");
terminalId = teclado.nextInt();

System.out.print("Temperatura del sensor (°C): ");
tempVestibulo = teclado.nextDouble();

System.out.print("Segundos de actividad del terminal (uptime): ");
segundosActividadTerminal = teclado.nextInt();

teclado.nextLine();

System.out.print("DNI de la persona: ");
dniPersona = teclado.nextLine();

System.out.print("Nombre completo: ");
nombrePersona = teclado.nextLine();

System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
perfilPersona = teclado.next().charAt(0);

esEntrada = true;

System.out.print("Hora de entrada (0-23): ");
horaEntrada = teclado.nextInt();

System.out.print("Minuto de entrada (0-59): ");
minutoEntrada = teclado.nextInt();

System.out.print("Hora de salida (0-23): ");
horaSalida = teclado.nextInt();

System.out.print("Minuto de salida (0-59): ");
minutoSalida = teclado.nextInt();

minutosTotalesEntrada = (horaEntrada * 60) + minutoEntrada;
minutosTotalesSalida = (horaSalida * 60) + minutoSalida;
minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

personasEnCentro++;
aforoDisponible--;
idUltimoFichaje++;

tokenResumen = dniPersona;
tokenResumen += "-T" + terminalId;
tokenResumen += "-ESTANCIA-" + minutosEstanciaTotal;
tokenResumen += "-REG" + idUltimoFichaje;

horasUptime = segundosActividadTerminal / 3600;
minutosUptime = (segundosActividadTerminal % 3600) / 60;
segundosUptime = segundosActividadTerminal % 60;

System.out.println("Terminal configurado: #" + terminalId);
System.out.println("Lectura térmica: " + tempVestibulo + " °C");
System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
System.out.println("Perfil: " + perfilPersona);
System.out.println("Sentido del paso: Entrada (" + esEntrada + ")");
System.out.println("Token: " + tokenResumen);
System.out.println("Horario: Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
System.out.println("Permanencia total en centro: " + minutosEstanciaTotal + " minutos.");
System.out.println("Estado actual del aforo:");
System.out.println("- personas en el centro: " + personasEnCentro + " (+1)");
System.out.println("- plazas libres: " + aforoDisponible + " (-1)");
System.out.println(horasUptime + " horas, " + minutosUptime + " minutos y " + segundosUptime + " segundos en servicio.");

teclado.close();
}
}

```

4. Trabajo del estudiante: evolución de su proyecto propio

El estudiante abre su archivo `.java` (que estaba en v0.4):

- Aplica la división entera `/` y el operador módulo `%` para descomponer una magnitud adaptada a su proyecto:
- En **Aventura conversacional**: descomponer el tiempo total de travesía en días completos y horas restantes, o las monedas totales en monedas de plata y cobre sobrante.
- En **Motor de recomendación**: descomponer la duración total de un contenido multimedia en horas completas y minutos restantes.
- En **Simulador de físicas 2D**: descomponer el tiempo de simulación en segundos completos y milisegundos restantes, o fotogramas en segundos y frames sueltos.
- En **Bóveda de contraseñas**: descomponer los días de vigencia restantes de una clave en semanas completas y días sueltos.
- Muestra el desglose por consola y comprueba con la calculadora que la reconstrucción matemática es perfecta.

2.2.3 Día 7 - 2 sesiones

1. Caso guía en AzaharTech

Es miércoles por la tarde. En el laboratorio de pruebas, **Pau Ferrer** muestra la nueva funcionalidad que ha intentado añadir a `ControlAccesoQR`: calcular el porcentaje de alumnos que han entrado a primera hora respecto al aforo total del vestíbulo.

Ha introducido:

- Personas en el centro: 1201
- Aforo total: 2000

Y ha escrito en el código:

```
double porcentajeOcupacion = (personasEnCentro / aforoTotal) * 100;
```

Al ejecutarlo, la consola imprime:

```
Porcentaje de ocupación: 0.0 %
```

Pau se lleva las manos a la cabeza:

«¡Pero si hay más de 1200 personas de 2000 plazas! ¡Eso es un 60 % de ocupación! ¿Por qué Java dice cero coma cero?».

Alba Torres y **Laia Claramunt** se acercan a la pantalla. Laia explica:

«Has caído en la trampa del tipado estático. `personasEnCentro` y `aforoTotal` son dos variables `int`. Java evalúa los paréntesis de izquierda a derecha: divide `1201 / 2000`, y como ambos son enteros, trunca los decimales y da `0`. Después multiplica `0 * 100`, que da `0`. Y solo al final, al guardarlo en la variable `double`, lo convierte en `0.0`.

Para solucionar esto debemos aplicar un casting explícito (`double`), forzando a la CPU a trabajar en coma flotante desde el primer paso de la división.

Hoy evolucionaremos `ControlAccesoQR` a la versión v0.6: aprenderemos a blindar la precedencia matemática con paréntesis, calcularemos porcentajes reales y aplicaremos el casting inverso a (`int`) para extraer la parte entera».

2.2.4 2. Precedencia de operadores y casting en Java

TABLA DE PRECEDENCIA DE EVALUACIÓN EN JAVA		
Prioridad	Operadores	Dirección de evaluación
1. Paréntesis	()	De dentro hacia afuera
2. Unarios	++, --, +, -, (tipo) [Casting]	De derecha a izquierda
3. Aritmética	*, /, %	De izquierda a derecha
4. Aritmética	+, -	De izquierda a derecha
5. Asignación	=, +=, -=, *=, /=, %=	De derecha a izquierda

A. LAS CONVERSIONES DE TIPO EN LA MEMORIA RAM

1. Conversión implícita (ensanchamiento / *widening*):

- Ocurre de forma automática y segura cuando un tipo de menor tamaño se almacena en uno mayor (`int` $\rightarrow$ `double`). No hay riesgo de pérdida de información.

2. Conversión explícita (estrechamiento / *narrowing* / *casting*):

- Es obligatoria cuando el programador fuerza la conversión de un tipo de mayor precisión en uno menor (`double` $\rightarrow$ `int`).
- **Se produce truncamiento:** Los decimales se eliminan por completo (no se redondean).
- Se antepone el tipo destino entre paréntesis:

```
double porcentaje = 60.05;
int porcentajeEntero = (int) porcentaje; // Almacena 60 (pierde .05)
```

B. LA SOLUCIÓN TÉCNICA MEDIANTE CASTING EN DIVISIONES

Para calcular el porcentaje sin perder decimales en la división entera, forzamos que al menos una variable sea tratada como decimal:

```
double porcentajeOcupacion = ((double) personasEnCentro / aforoTotal) * 100.0;
```

Al aplicar `(double)` sobre `personasEnCentro`, la división pasa a ser de tipo `double / int`, lo que promociona toda la operación a coma flotante y devuelve el **60.05 %** exacto.

3. Evolución a `ControlAccesoQR v0.6`

Ampliamos el archivo incorporando el aforo total de 2000 plazas, el cálculo de porcentaje exacto con casting y el truncamiento a valor entero.

PASO A. EVOLUCIÓN EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — V0.6)

```
Algoritmo ControlAccesoQR
    Definir terminalId Como Entero
    Definir tempVestibulo Como Real
    Definir nombrePersona, dniPersona Como Cadena
    Definir perfilPersona Como Caracter
    Definir esEntrada Como Logico

    Definir horaEntrada, minutoEntrada Como Entero
    Definir horaSalida, minutoSalida Como Entero
    Definir minutosTotalesEntrada Como Entero
    Definir minutosTotalesSalida Como Entero
    Definir minutosEstanciaTotal Como Entero
    Definir tokenResumen Como Cadena

    Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
    personasEnCentro <- 1200
    aforoDisponible <- 800
    idUltimoFichaje <- 1042

    Definir segundosActividadTerminal Como Entero
    Definir horasUptime, minutosUptime, segundosUptime Como Entero

    Definir aforoTotal Como Entero
    Definir porcentajeOcupacionReal Como Real
    Definir porcentajeOcupacionEntero Como Entero

    Escribir "ID del terminal:"
    Leer terminalId
    Escribir "Temperatura del sensor (°C):"
    Leer tempVestibulo
    Escribir "Segundos de actividad del terminal (uptime):"
    Leer segundosActividadTerminal

    Escribir "DNI de la persona:"
    Leer dniPersona
    Escribir "Nombre completo:"
    Leer nombrePersona
    Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
    Leer perfilPersona

    esEntrada <- Verdadero

    Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
    Leer horaEntrada
    Leer minutoEntrada
    Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
    Leer horaSalida
    Leer minutoSalida

    minutosTotalesEntrada <- (horaEntrada * 60) + minutoEntrada
    minutosTotalesSalida <- (horaSalida * 60) + minutoSalida
```

```

minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

personasEnCentro <- personasEnCentro + 1
aforoDisponible <- aforoDisponible - 1
idUltimoFichaje <- idUltimoFichaje + 1

tokenResumen <- dniPersona
tokenResumen <- tokenResumen + "-T" + ConvertirATexto(terminalId)
tokenResumen <- tokenResumen + "-ESTANCIA-" + ConvertirATexto(minutosEstanciaTotal)
tokenResumen <- tokenResumen + "-REG" + ConvertirATexto(idUltimoFichaje)

horasUptime <- trunc(segundosActividadTerminal / 3600)
minutosUptime <- trunc((segundosActividadTerminal MOD 3600) / 60)
segundosUptime <- segundosActividadTerminal MOD 60

aforoTotal <- personasEnCentro + aforoDisponible
porcentajeOcupacionReal <- (personasEnCentro * 100.0) / aforoTotal
porcentajeOcupacionEntero <- trunc(porcentajeOcupacionReal)

Escribir "Terminal configurado: #", terminalId
Escribir "Lectura térmica: ", tempVestibulo, " °C"
Escribir "Persona: ", nombrePersona, " (DNI: ", dniPersona, ")"
Escribir "Perfil: ", perfilPersona
Escribir "Sentido del paso: Entrada (", esEntrada, ")"
Escribir "Token: ", tokenResumen
Escribir "Horario: Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
Escribir "Permanencia total en centro: ", minutosEstanciaTotal, " minutos."
Escribir "Diagnóstico del terminal (uptime):"
Escribir horasUptime, " horas, ", minutosUptime, " minutos y ", segundosUptime, " segundos en servicio."
Escribir "Estado del aforo (capacidad: ", aforoTotal, " plazas):"
Escribir "- personas en el centro: ", personasEnCentro, " (+1)"
Escribir "- plazas libres: ", aforoDisponible, " (-1)"
Escribir "- ocupación exacta: ", porcentajeOcupacionReal, " %"
Escribir "- ocupación en panel: ", porcentajeOcupacionEntero, " %"
FinAlgoritmo

```

PASO B. EVOLUCIÓN EN JAVA (pr/src/ControlAccesoQR.java — V0.6)

Abrimos nuestro archivo maestro `ControlAccesoQR.java` y lo evolucionamos a v0.6:

```

import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;
        String nombrePersona, dniPersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
        int minutosEstanciaTotal;
        String tokenResumen;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionEntero;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (°C): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();

        System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
        perfilPersona = teclado.next().charAt(0);

        esEntrada = true;
    }
}

```

```

System.out.print("Hora de entrada (0-23): ");
horaEntrada = teclado.nextInt();

System.out.print("Minuto de entrada (0-59): ");
minutoEntrada = teclado.nextInt();

System.out.print("Hora de salida (0-23): ");
horaSalida = teclado.nextInt();

System.out.print("Minuto de salida (0-59): ");
minutoSalida = teclado.nextInt();

minutosTotalesEntrada = (horaEntrada * 60) + minutoEntrada;
minutosTotalesSalida = (horaSalida * 60) + minutoSalida;
minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

personasEnCentro++;
aforoDisponible--;
idUltimoFichaje++;

tokenResumen = dniPersona;
tokenResumen += "-T" + terminalId;
tokenResumen += "-ESTANCIA-" + minutosEstanciaTotal;
tokenResumen += "-REG" + idUltimoFichaje;

horasUptime = segundosActividadTerminal / 3600;
minutosUptime = (segundosActividadTerminal % 3600) / 60;
segundosUptime = segundosActividadTerminal % 60;

aforoTotal = personasEnCentro + aforoDisponible;
porcentajeOcupacionReal = ((double) personasEnCentro / aforoTotal) * 100.0;
porcentajeOcupacionEntero = (int) porcentajeOcupacionReal;

System.out.println("Terminal configurado: #" + terminalId);
System.out.println("Lectura térmica: " + tempVestibulo + " °C");
System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
System.out.println("Perfil: " + perfilPersona);
System.out.println("Sentido del paso: Entrada (" + esEntrada + ")");
System.out.println("Token: " + tokenResumen);
System.out.println("Horario: Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
System.out.println("Permanencia total en centro: " + minutosEstanciaTotal + " minutos.");
System.out.println("Diagnóstico del terminal (uptime):");
System.out.println(horasUptime + " horas, " + minutosUptime + " minutos y " + segundosUptime + " segundos en servicio.");
System.out.println("Estado del aforo (capacidad: " + aforoTotal + " plazas):");
System.out.println("- personas en el centro: " + personasEnCentro + " (+1)");
System.out.println("- plazas libres: " + aforoDisponible + " (-1)");
System.out.println("- ocupación exacta: " + porcentajeOcupacionReal + "%");
System.out.println("- ocupación en panel: " + porcentajeOcupacionEntero + "%");

teclado.close();
}
}

```

4. Trabajo del estudiante: evolución de su proyecto propio

El estudiante abre su archivo `.java` (en versión v0.5):

- Incorpora el cálculo de un ratio o porcentaje que divida dos variables enteras adaptadas a su proyecto, aplicando casting explícito (double) y casting inverso a entero (int):
- En Aventura conversacional: calcular el porcentaje exacto de salud restante respecto al total máximo y truncar a (int) para dibujar una barra de vida.
- En Motor de recomendación: calcular el porcentaje de afinidad sobre la puntuación máxima teórica y truncar a (int) para mostrar el nivel de coincidencia.
- En Simulador de físicas 2D: calcular el porcentaje de energía cinética conservada tras el rebote y truncar a (int).
- En Bóveda de contraseñas: calcular el porcentaje de vigencia consumido de la credencial respecto a su vida útil total y extraer la parte entera con (int).
- Comprueba con la calculadora que la división decimal no trunca a cero y que la parte entera descarta los decimales correctamente.

2.2.5 Día 8 - 2 sesiones

1. Caso guía en AzaharTech

Es jueves por la tarde. Concluyen las primeras dos semanas del sprint. **Laia Claramunt** revisa en la pantalla de la sala el avance global:

«Hoy cerramos formalmente la Semana 2. En la Semana 1 creamos el esqueleto y la identidad de nuestro software. Durante estos últimos cuatro días habéis convertido el programa en un auténtico motor matemático: domináis los operadores compuestos (+=), la descomposición con módulo (%) y las conversiones de tipo con casting.

Hoy dedicaremos estas dos horas a auditar y afianzar la versión v0.6 de vuestro proyecto propio de la bolsa de proyectos. No debe quedar ni un solo cálculo ambiguo ni una división entera accidental.

Al sonar el timbre, la versión v0.6 de vuestra aplicación debe estar confirmada y subida a GitHub».

2. Lista de comprobación técnico de calidad del código para la versión v0.6

Antes de realizar el commit, cada estudiante debe verificar los siguientes 5 puntos en IntelliJ:

1. **Ejecución secuencial.** El código se ejecuta de forma estrictamente secuencial de principio a fin.
2. **Casting explícito justificado.** Al menos una división entre enteros cuenta con `(double)` para preservar la precisión decimal.
3. **Uso del operador módulo (%).** Al menos una magnitud se descompone o calcula mediante el residuo de una división.
4. **Protección con paréntesis () .** Las fórmulas complejas están agrupadas con paréntesis para garantizar la precedencia matemática sin ambigüedades.
5. **Indentación automática.** Se ha pulsado `Ctrl + Alt + L` para ordenar el código según el estándar de estilo oficial.

2.2.6 3. La versión v0.6 del proyecto propio del estudiante

Cada estudiante comprueba que su clase única `pr/src/MiProyecto.java` ha evolucionado de forma acumulativa e incremental tanto en pseudocódigo como en Java:

```
Algoritmo ControlAccesoQR
  Definir terminalId Como Entero
  Definir tempVestibulo Como Real
  Definir nombrePersona, dniPersona Como Cadena
  Definir perfilPersona Como Caracter
  Definir esEntrada Como Logico

  Definir horaEntrada, minutoEntrada Como Entero
  Definir horaSalida, minutoSalida Como Entero
  Definir minutosTotalesEntrada Como Entero
  Definir minutosTotalesSalida Como Entero
  Definir minutosEstanciaTotal Como Entero
  Definir tokenResumen Como Cadena

  Definir personasEnCentro, aforoDisponible, idUltimoFichaje Como Entero
  personasEnCentro <- 1200
  aforoDisponible <- 800
  idUltimoFichaje <- 1042

  Definir segundosActividadTerminal Como Entero
  Definir horasUptime, minutosUptime, segundosUptime Como Entero

  Definir aforoTotal Como Entero
  Definir porcentajeOcupacionReal Como Real
  Definir porcentajeOcupacionEntero Como Entero

  Escribir "ID del terminal:"
  Leer terminalId
  Escribir "Temperatura del sensor (°C):"
  Leer tempVestibulo
  Escribir "Segundos de actividad del terminal (uptime):"
  Leer segundosActividadTerminal

  Escribir "DNI de la persona:"
  Leer dniPersona
  Escribir "Nombre completo:"
  Leer nombrePersona
  Escribir "Perfil de acceso (E = Estudiante, D = Docente, V = Visita):"
  Leer perfilPersona

  esEntrada <- Verdadero
```

```

Escribir "Introduce hora y minuto de entrada (por ejemplo, 8 15):"
Leer horaEntrada
Leer minutoEntrada
Escribir "Introduce hora y minuto de salida (por ejemplo, 14 10):"
Leer horaSalida
Leer minutoSalida

minutosTotalesEntrada <- (horaEntrada * 60) + minutoEntrada
minutosTotalesSalida <- (horaSalida * 60) + minutoSalida
minutosEstanciaTotal <- minutosTotalesSalida - minutosTotalesEntrada

personasEnCentro <- personasEnCentro + 1
aforoDisponible <- aforoDisponible - 1
idUltimoFichaje <- idUltimoFichaje + 1

tokenResumen <- dniPersona
tokenResumen <- tokenResumen + "-T" + ConvertirATexto(terminalId)
tokenResumen <- tokenResumen + "-ESTANCIA-" + ConvertirATexto(minutosEstanciaTotal)
tokenResumen <- tokenResumen + "-REG" + ConvertirATexto(idUltimoFichaje)

horasUptime <- trunc(segundosActividadTerminal / 3600)
minutosUptime <- trunc((segundosActividadTerminal MOD 3600) / 60)
segundosUptime <- segundosActividadTerminal MOD 60

aforoTotal <- personasEnCentro + aforoDisponible
porcentajeOcupacionReal <- (personasEnCentro * 100.0) / aforoTotal
porcentajeOcupacionEntero <- trunc(porcentajeOcupacionReal)

Escribir "Terminal configurado: #", terminalId
Escribir "Lectura térmica: ", tempVestibulo, " °C"
Escribir "Persona: ", nombrePersona, " (DNI: ", dniPersona, ")"
Escribir "Perfil: ", perfilPersona
Escribir "Sentido del paso: Entrada (" , esEntrada, ")"
Escribir "Token: ", tokenResumen
Escribir "Horario: Entrada ", horaEntrada, ":", minutoEntrada, " | Salida ", horaSalida, ":", minutoSalida
Escribir "Permanencia total en centro: ", minutosEstanciaTotal, " minutos."
Escribir "Diagnóstico del terminal (uptime):"
Escribir horasUptime, " horas, ", minutosUptime, " minutos y ", segundosUptime, " segundos en servicio."
Escribir "Estado del aforo (capacidad: ", aforoTotal, " plazas):"
Escribir "- personas en el centro: ", personasEnCentro, " (+1)"
Escribir "- plazas libres: ", aforoDisponible, " (-1)"
Escribir "- ocupación exacta: ", porcentajeOcupacionReal, " %"
Escribir "- ocupación en panel: ", porcentajeOcupacionEntero, " %"
FinAlgoritmo

```

```

import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;
        String nombrePersona, dniPersona;
        char perfilPersona;
        boolean esEntrada;

        int horaEntrada, minutoEntrada;
        int horaSalida, minutoSalida;
        int minutosTotalesEntrada;
        int minutosTotalesSalida;
        int minutosEstanciaTotal;
        String tokenResumen;

        int personasEnCentro = 1200;
        int aforoDisponible = 800;
        int idUltimoFichaje = 1042;

        int aforoTotal;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionEntero;

        int segundosActividadTerminal;
        int horasUptime, minutosUptime, segundosUptime;

        System.out.print("ID del terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura del sensor (°C): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos de actividad del terminal (uptime): ");
        segundosActividadTerminal = teclado.nextInt();

        teclado.nextLine();

        System.out.print("DNI de la persona: ");
        dniPersona = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombrePersona = teclado.nextLine();
    }
}

```

```

System.out.print("Perfil de acceso (E = Estudiante, D = Docente, V = Visita): ");
perfilPersona = teclado.next().charAt(0);

esEntrada = true;

System.out.print("Hora de entrada (0-23): ");
horaEntrada = teclado.nextInt();

System.out.print("Minuto de entrada (0-59): ");
minutoEntrada = teclado.nextInt();

System.out.print("Hora de salida (0-23): ");
horaSalida = teclado.nextInt();

System.out.print("Minuto de salida (0-59): ");
minutoSalida = teclado.nextInt();

minutosTotalesEntrada = (horaEntrada * 60) + minutoEntrada;
minutosTotalesSalida = (horaSalida * 60) + minutoSalida;
minutosEstanciaTotal = minutosTotalesSalida - minutosTotalesEntrada;

personasEnCentro++;
aforoDisponible--;
idUltimoFichaje++;

tokenResumen = dniPersona;
tokenResumen += "-T" + terminalId;
tokenResumen += "-ESTANCIA-" + minutosEstanciaTotal;
tokenResumen += "-REG" + idUltimoFichaje;

horasUptime = segundosActividadTerminal / 3600;
minutosUptime = (segundosActividadTerminal % 3600) / 60;
segundosUptime = segundosActividadTerminal % 60;

aforoTotal = personasEnCentro + aforoDisponible;
porcentajeOcupacionReal = ((double) personasEnCentro / aforoTotal) * 100.0;
porcentajeOcupacionEntero = (int) porcentajeOcupacionReal;

System.out.println("Terminal configurado: #" + terminalId);
System.out.println("Lectura térmica: " + tempVestibulo + " °C");
System.out.println("Persona: " + nombrePersona + " (DNI: " + dniPersona + ")");
System.out.println("Perfil: " + perfilPersona);
System.out.println("Sentido del paso: Entrada (" + esEntrada + ")");
System.out.println("Token: " + tokenResumen);
System.out.println("Horario: Entrada " + horaEntrada + ":" + minutoEntrada + " | Salida " + horaSalida + ":" + minutoSalida);
System.out.println("Permanencia total en centro: " + minutosEstanciaTotal + " minutos.");
System.out.println("Diagnóstico del terminal (uptime):");
System.out.println(horasUptime + " horas, " + minutosUptime + " minutos y " + segundosUptime + " segundos en servicio.");
System.out.println("Estado del aforo (capacidad: " + aforoTotal + " plazas):");
System.out.println("- personas en el centro: " + personasEnCentro + " (+1)");
System.out.println("- plazas libres: " + aforoDisponible + " (-1)");
System.out.println("- ocupación exacta: " + porcentajeOcupacionReal + "%");
System.out.println("- ocupación en panel: " + porcentajeOcupacionEntero + "%");

teclado.close();
}
}

```

4. Cierre formal en Git y sincronización con GitHub

El estudiante actualiza su repositorio con los avances de la segunda semana desde la interfaz de IntelliJ IDEA:

1. Abre el panel lateral **Commit** de IntelliJ (Alt + 0 o Ctrl + K).
2. Marca la casilla de los archivos modificados (pr/pseudocodigo/MiProyecto.psc y pr/src/MiProyecto.java) para pasarlos al área de preparación (*Staged*).
3. Escribe en el cuadro de texto el mensaje siguiendo el estándar convencional:

```
feat(pr): evolucionar aplicacion propia a v0.6 con motor matematico, modulo y casting
```

4. Despliega el botón de confirmación y selecciona **Commit and Push...** (o pulsa Ctrl + Alt + K).
5. En la ventana emergente, pulsa **Push** para enviar los cambios al repositorio remoto en GitHub.
6. Abre el navegador web y comprueba que el commit y ambos archivos aparecen actualizados en tu repositorio.

2.3 Semana 3. La arquitectura del código - constantes, escapes y printf

Llegamos a la semana final del Sprint 1. Partiendo de la versión `ControlAccesoQR v0.6` (que ya cuenta con captura completa, descomposición con módulo y casting), evolucionamos nuestro archivo maestro eliminando números mágicos (`v0.7`), maquetando con secuencias de escape (`v0.8`), formateando con `printf` (`v0.9`) y sellando la versión definitiva `v1.0` secuencial con comentarios formales, mientras el estudiante concluye paralelamente su archivo único `MiProyecto.java`.

2.3.1 Día 9 - 2 sesiones

1. Caso guía en AzaharTech

Es lunes por la tarde. En la sala técnica de **AzaharTech**, **Alba Torres** proyecta en el monitor principal la clase `ControlAccesoQR.java` tal y como quedó el jueves anterior (versión `v0.6`). Con el cursor, resalta varios números y textos dispersos por las líneas de cálculo:

```
int minutosTotalesLectivos = (totalSesiones * 50) + minutosExtraGuardia;
horasUptime =segundosActividadTerminal /3600;
minutosUptime =(segundosActividadTerminal %3600)/60;
porcentajeOcupacionReal =((double)contadorFichajesTerminal /aforoMaximoVestibulo)*100.0;
```

Alba se dirige a **Pau Ferrer** y al estudiante:

«Fijaos en esos números: `50`, `3600`, `60`, `100.0`. En la ingeniería de software profesional los llamamos números mágicos (magic numbers): valores fijos incrustados directamente en mitad de las operaciones matemáticas.

¿Qué ocurre si la dirección del IES El Caminàs decide cambiar la duración de las sesiones lectivas a 55 minutos el curso que viene? Tendríamos que buscar ese `50` en mitad de las fórmulas, arriesgándonos a cambiar un dato por error.

Hoy abriremos nuestro archivo `ControlAccesoQR` y lo refactorizaremos a la versión `v0.7`: extraeremos todos los valores fijos y los convertiremos en constantes inmutables protegidas por el compilador con la palabra clave `final` al inicio del método».

2. Fundamento teórico: constantes inmutables y literales tipados

ELIMINACIÓN DE NÚMEROS MÁGICOS EN MEMORIA		
Código Frágil (Hardcoded)	Código Profesional (final)	Ventaja de Ingeniería
total * 50; segundos / 3600; total * 100.0;	total * MINUTOS_SESION; segundos / SEGUNDOS_HORA; total * FACTOR_PORCENTAJE;	Si cambia el valor, solo se modifica en una única línea de configuración al inicio.

- La palabra reservada `final`.** Le indica al compilador que la posición de memoria es de solo lectura. Cualquier intento de reasignar su valor provocará un error de compilación.
- Convención `UPPER_SNAKE_CASE`.** Todas las letras en mayúsculas separadas por guiones bajos (`_`). Permite que cualquier miembro del equipo identifique al instante que se trata de un valor inmutable.
- Literales numéricos tipados.** El sufijo `L` para enteros largos (`long`) y `F` para decimales simples (`float`).

3. Refactorización a `ControlAccesoQR v0.7`

Abrimos el archivo y extraemos todos los números fijos a la cabecera de la clase.

PASO A. REFACTORIZACIÓN EN PSEINT (`pr/pseudocodigo/ControlAccesoQR.psc` — `V0.7`)

```
Algoritmo ControlAccesoQR
// =====
// SISTEMA DE CONTROL DE ASISTENCIA QR - IES EL CAMINAS (Castellon)
// Version: 0.7 (Evolucion: Extraccion de Constantes Inmutables)
```

```
// =====

// [NUEVO DÍA 9] Declaracion centralizada de constantes de configuracion
Definir NOMBRE_INSTITUTO, PREFIJO_CENTRO Como Cadena
Definir MINUTOS_POR_SESION, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
Definir FACTOR_PORCENTAJE Como Real

NOMBRE_INSTITUTO <- "IES El Caminas (Castellon)"
PREFIJO_CENTRO <- "CAMINAS"
MINUTOS_POR_SESION <- 50
SEGUNDOS_POR_HORA <- 3600
SEGUNDOS_POR_MINUTO <- 60
FACTOR_PORCENTAJE <- 100.0

// Variables de memoria
Definir terminalId, contadorFichajesTerminal, aforoMaximoVestibulo Como Entero
Definir tempVestibulo, porcentajeOcupacionReal Como Real
Definir porcentajeOcupacionTruncado Como Entero
Definir nombreEstudiante, dniEstudiante, tokenResumen Como Cadena
Definir letraGrupo Como Caracter
Definir matriculaActiva Como Logico
Definir sesionesManana, sesionesTarde, totalSesiones, minutosTotalesLectivos Como Entero
Definir minutosExtraGuardia, segundosActividadTerminal Como Entero
Definir horasUptime, minutosUptime, segundosUptime Como Entero

contadorFichajesTerminal <- 0

// Captura de datos
Escribir "=== AZAHARTECH: TERMINAL " + NOMBRE_INSTITUTO + " (v0.7) ==="
Escribir "ID Terminal, temperatura y segundos de actividad:"
Leer terminalId
Leer tempVestibulo
Leer segundosActividadTerminal

Escribir "Aforo maximo del vestibulo:"
Leer aforoMaximoVestibulo

Escribir "DNI, nombre y grupo del alumno:"
Leer dniEstudiante
Leer nombreEstudiante
Leer letraGrupo
matriculaActiva <- Verdadero

Escribir "Sesiones de manana, tarde y minutos de guardia:"
Leer sesionesManana
Leer sesionesTarde
Leer minutosExtraGuardia

// Procesamiento usando exclusivamente constantes
contadorFichajesTerminal <- contadorFichajesTerminal + 1
totalSesiones <- sesionesManana + sesionesTarde
minutosTotalesLectivos <- (totalSesiones * MINUTOS_POR_SESION) + minutosExtraGuardia

horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

porcentajeOcupacionReal <- (contadorFichajesTerminal * FACTOR_PORCENTAJE) / aforoMaximoVestibulo
porcentajeOcupacionTruncado <- trunc(porcentajeOcupacionReal)

tokenResumen <- PREFIJO_CENTRO + "-" + dniEstudiante + "-T" + ConvertirATexto(terminalId)

// Salida consolidada
Escribir "=====
Escribir "CENTRO:      ", NOMBRE_INSTITUTO
Escribir "FICHAJE:    #", contadorFichajesTerminal, " | TOKEN: ", tokenResumen
Escribir "ESTUDIANTE: ", nombreEstudiante, " (", letraGrupo, " DAM)"
Escribir "PERMANENCIA:", minutosTotalesLectivos, " min lectivos."
Escribir "AFORO:      ", porcentajeOcupacionReal, " % (Panel: ", porcentajeOcupacionTruncado, " %)"
Escribir "UPTIME:      ", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s"
Escribir "=====
FinAlgoritmo
```

PASO B. REFACTORIZACIÓN EN JAVA (pr/src/ControlAccesoQR.java — V0.7)

```
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 0.7: Parametrización con constantes inmutables ('final') y eliminación de números mágicos.
 * Módulo: Programación (PR) - Sprint 1 (RA1)
 */

import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        // -----
        // 1. CONSTANTES INMUTABLES DEL SISTEMA (UPPER_SNAKE_CASE)
        // -----
        final String NOMBRE_INSTITUTO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
```

```

final int MINUTOS_POR_SESION = 50;
final int SEGUNDOS_POR_HORA = 3600;
final int SEGUNDOS_POR_MINUTO = 60;
final double FACTOR_PORCENTAJE = 100.0;

// -----
// 2. VARIABLES DE MEMORIA
// -----
Scanner teclado = new Scanner(System.in);

int terminalId;
double tempVestibulo;
int contadorFichajesTerminal = 0;
int segundosActividadTerminal;
int aforoMaximoVestibulo;

String dniEstudiante;
String nombreEstudiante;
char letraGrupo;
boolean matriculaActiva = true;

int sesionesManana;
int sesionesTarde;
int totalSesiones;
int minutosExtraGuardia;
int minutosTotalesLectivos;

int horasUptime;
int minutosUptime;
int segundosUptime;
double porcentajeOcupacionReal;
int porcentajeOcupacionTruncado;
String tokenResumen;

// -----
// 3. ENTRADA DE DATOS
// -----
System.out.println("=====");
System.out.println("  AZAHARTECH - TERMINAL " + NOMBRE_INSTITUTO);
System.out.println("  Versión 0.7 (Parámetros Inmutables Activos)  ");
System.out.println("=====");
System.out.print("ID Terminal: ");
terminalId = teclado.nextInt();

System.out.print("Temperatura sensor (°C): ");
tempVestibulo = teclado.nextDouble();

System.out.print("Segundos acumulados de actividad: ");
segundosActividadTerminal = teclado.nextInt();

System.out.print("Aforo máximo permitido en vestíbulo: ");
aforoMaximoVestibulo = teclado.nextInt();

teclado.nextLine(); // Limpieza obligatoria del buffer

System.out.print("DNI Estudiante: ");
dniEstudiante = teclado.nextLine();

System.out.print("Nombre completo: ");
nombreEstudiante = teclado.nextLine();

System.out.print("Grupo (letra): ");
letraGrupo = teclado.next().charAt(0);

System.out.print("Sesiones turno mañana: ");
sesionesManana = teclado.nextInt();

System.out.print("Sesiones turno tarde: ");
sesionesTarde = teclado.nextInt();

System.out.print("Minutos de guardia/tutoría: ");
minutosExtraGuardia = teclado.nextInt();

// -----
// 4. PROCESAMIENTO SECUENCIAL USANDO CONSTANTES
// -----
contadorFichajesTerminal++;
totalSesiones = sesionesManana + sesionesTarde;
minutosTotalesLectivos = (totalSesiones * MINUTOS_POR_SESION) + minutosExtraGuardia;

// Descomposición horaria con constantes inmutables
horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

// Cálculo de porcentaje con casting y factor porcentual constante
porcentajeOcupacionReal = ((double) contadorFichajesTerminal / aforoMaximoVestibulo) * FACTOR_PORCENTAJE;
porcentajeOcupacionTruncado = (int) porcentajeOcupacionReal;

tokenResumen = PREFIJO_CENTRO + "-" + dniEstudiante + "-T" + terminalId;

// -----
// 5. SALIDA CONSOLIDADA
// -----

```

```

System.out.println("=====");
System.out.println("CENTRO:      " + NOMBRE_INSTITUTO);
System.out.println("FICHAJE:    #" + contadorFichajesTerminal + " | TOKEN: " + tokenResumen);
System.out.println("ESTUDIANTE: " + nombreEstudiante + " (" + letraGrupo + " DAM)");
System.out.println("PERMANENCIA: " + minutosTotalesLectivos + " min lectivos.");
System.out.println("AFORO:      " + porcentajeOcupacionReal + " % (Panel: " + porcentajeOcupacionTruncado + " %)");
System.out.println("UPTIME:     " + horasUptime + "h " + minutosUptime + "m " + segundosUptime + "s");
System.out.println("=====");

teclado.close();
}
}

```

4. Trabajo del estudiante: refactorización de su proyecto propio

El estudiante abre su archivo maestro `.java` (versión v0.6):

- Extrae todos los números y cadenas fijas a constantes `final` al inicio del método `main`.
- Sustituye en todas las operaciones del programa los literales sueltos por los identificadores de constantes en mayúsculas (`UPPER_SNAKE_CASE`).
- Compila y comprueba que el programa se ejecuta de forma idéntica, pero con una mantenibilidad infinitamente superior.

2.4 ---

2.4.1 Día 10 - 2 sesiones

1. Caso guía en AzaharTech

Es martes por la tarde. **Pau Ferrer** muestra la consola de IntelliJ con la versión v0.7 en ejecución:

«El cálculo con constantes funciona de maravilla, pero la salida sigue teniendo un aspecto descuidado: para separar bloques tengo que poner `System.out.println("");` repetidas veces y las palabras 'TOKEN', 'CENTRO' y 'PERMANENCIA' no están alineadas porque cada palabra tiene una longitud distinta».

Laia Claramunt y **Alba Torres** le indican la barra invertida (`\`):

«Hoy evolucionaremos `ControlAccesoQR.java` a la versión v0.8. Utilizaremos secuencias de escape: introduciremos tabuladores horizontales (`\t`) para crear columnas alineadas, saltos de línea (`\n`) para separar bloques en una sola instrucción y escaparemos comillas dobles (`\"`) para citar el protocolo oficial del centro educativo».

2. Fundamento teórico: secuencias de escape en cadenas Java

SECUENCIAS DE ESCAPE EN CADENAS DE TEXTO		
Secuencia	Nombre técnico	Efecto en la consola de salida
<code>\n</code>	Salto de línea	Pasa a la siguiente línea inmediatamente.
<code>\t</code>	Tabulación horizontal	Avanza hasta la siguiente parada de columna.
<code>\"</code>	Comilla doble	Permite imprimir comillas dentro de un String.
<code>\\</code>	Barra invertida	Imprime el carácter literal de barra <code>\</code>

2.4.2.3. Evolución a `ControlAccesoQR v0.8`

Modificamos el bloque de salida del archivo maestro `ControlAccesoQR` incorporando las secuencias de escape.

PASO A. EVOLUCIÓN EN PSEINT ([pr/pseudocodigo/ControlAccesoQR.psc](#) — V0.8)

```

Algoritmo ControlAccesoQR
// =====
// SISTEMA DE CONTROL DE ASISTENCIA QR - IES EL CAMINAS (Castellon)
// Version: 0.8 (Evolucion: Secuencias de Escape y Formato de Consola)
// =====

// Constantes
Definir NOMBRE_INSTITUTO, PREFIJO_CENTRO Como Cadena
Definir MINUTOS_POR_SESION, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
Definir FACTOR_PORCENTAJE Como Real

NOMBRE_INSTITUTO <- "IES El Caminas (Castellon)"
PREFIJO_CENTRO <- "CAMINAS"
MINUTOS_POR_SESION <- 50
SEGUNDOS_POR_HORA <- 3600
SEGUNDOS_POR_MINUTO <- 60
FACTOR_PORCENTAJE <- 100.0

// Variables
Definir terminalId, contadorFichajesTerminal, aforoMaximoVestibulo Como Entero
Definir tempVestibulo, porcentajeOcupacionReal Como Real
Definir porcentajeOcupacionTruncado Como Entero
Definir nombreEstudiante, dniEstudiante, tokenResumen Como Cadena
Definir letraGrupo Como Caracter
Definir matriculaActiva Como Logico
Definir sesionesManana, sesionesTarde, totalSesiones, minutosTotalesLectivos Como Entero
Definir minutosExtraGuardia, segundosActividadTerminal Como Entero
Definir horasUptime, minutosUptime, segundosUptime Como Entero

contadorFichajesTerminal <- 0

// Entrada de datos
Escribir "=== AZAHARTECH: TERMINAL IES EL CAMINAS (v0.8) ==="
Escribir "ID Terminal, temperatura y segundos de actividad:"
Leer terminalId
Leer tempVestibulo
Leer segundosActividadTerminal
Escribir "Aforo maximo del vestibulo:"
Leer aforoMaximoVestibulo

Escribir "DNI, nombre y grupo del alumno:"
Leer dniEstudiante
Leer nombreEstudiante
Leer letraGrupo
matriculaActiva <- Verdadero

Escribir "Sesiones de manana, tarde y minutos de guardia:"
Leer sesionesManana
Leer sesionesTarde
Leer minutosExtraGuardia

// Cálculos
contadorFichajesTerminal <- contadorFichajesTerminal + 1
totalSesiones <- sesionesManana + sesionesTarde
minutosTotalesLectivos <- (totalSesiones * MINUTOS_POR_SESION) + minutosExtraGuardia

horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

porcentajeOcupacionReal <- (contadorFichajesTerminal * FACTOR_PORCENTAJE) / aforoMaximoVestibulo
porcentajeOcupacionTruncado <- trunc(porcentajeOcupacionReal)

tokenResumen <- PREFIJO_CENTRO + "-" + dniEstudiante + "-T" + ConvertirATexto(terminalId)

// [NUEVO DÍA 10] Salida estructurada con tabuladores y saltos limpios
Escribir ""
Escribir "=====
Escribir "ENTIDAD:\t", NOMBRE_INSTITUTO
Escribir "SISTEMA:\t"Control de Acceso Dinamico QR""
Escribir "UBICACION:\tVestibulo Principal \t Edificio A"
Escribir "=====
Escribir "FICHAJE N.:\t#", contadorFichajesTerminal, "\t\tESTADO:\tVALIDO"
Escribir "ESTUDIANTE:\t", nombreEstudiante
Escribir "IDENTIFICADOR:\t", dniEstudiante, "\t\tGRUPO:\t", letraGrupo, " DAM"
Escribir "TOKEN QR:\t", tokenResumen
Escribir "-----"
Escribir "PERMANENCIA:\t", minutosTotalesLectivos, " minutos lectivos."
Escribir "OCUPACION:\t", porcentajeOcupacionReal, " % (Panel: ", porcentajeOcupacionTruncado, " %)"
Escribir "SERVICIO:\t", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s"
Escribir "=====
FinAlgoritmo

```

PASO B. EVOLUCIÓN EN JAVA (pr/src/ControlAccesoQR.java — V0.8)

Actualizamos directamente el bloque de salida de nuestro archivo maestro `ControlAccesoQR.java` :

```

/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *

```

```

* Versión 0.8: Maquetación visual mediante secuencias de escape (\n, \t, \", \\\).
* Módulo: Programación (PR) - Sprint 1 (RA1)
*/

import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        // Constantes inmutables
        final String NOMBRE_INSTITUTO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final String RUTA_LOGS = "C:\\caminas\\terminal\\logs";
        final int MINUTOS_POR_SESION = 50;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;

        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;
        int contadorFichajesTerminal = 0;
        int segundosActividadTerminal;
        int aforoMaximoVestibulo;

        String dniEstudiante;
        String nombreEstudiante;
        char letraGrupo;
        boolean matriculaActiva = true;

        int sesionesManana;
        int sesionesTarde;
        int totalSesiones;
        int minutosExtraGuardia;
        int minutosTotalesLectivos;

        int horasUptime;
        int minutosUptime;
        int segundosUptime;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionTruncado;
        String tokenResumen;

        // Entrada de datos
        System.out.println("=====");
        System.out.println("  AZAHARTECH - TERMINAL  " + NOMBRE_INSTITUTO);
        System.out.println("=====");
        System.out.print("ID Terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura sensor (°C): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos acumulados de actividad: ");
        segundosActividadTerminal = teclado.nextInt();

        System.out.print("Aforo máximo permitido en vestíbulo: ");
        aforoMaximoVestibulo = teclado.nextInt();

        teclado.nextLine(); // Limpieza de buffer

        System.out.print("DNI Estudiante: ");
        dniEstudiante = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombreEstudiante = teclado.nextLine();

        System.out.print("Grupo (letra): ");
        letraGrupo = teclado.next().charAt(0);

        System.out.print("Sesiones turno mañana: ");
        sesionesManana = teclado.nextInt();

        System.out.print("Sesiones turno tarde: ");
        sesionesTarde = teclado.nextInt();

        System.out.print("Minutos de guardia/tutoría: ");
        minutosExtraGuardia = teclado.nextInt();

        // Procesamiento
        contadorFichajesTerminal++;
        totalSesiones = sesionesManana + sesionesTarde;
        minutosTotalesLectivos = (totalSesiones * MINUTOS_POR_SESION) + minutosExtraGuardia;

        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        porcentajeOcupacionReal = ((double) contadorFichajesTerminal / aforoMaximoVestibulo) * FACTOR_PORCENTAJE;
        porcentajeOcupacionTruncado = (int) porcentajeOcupacionReal;

        tokenResumen = PREFIJO_CENTRO + "-" + dniEstudiante + "-T" + terminalId;

        // [NUEVO DÍA 10] Salida estructurada con secuencias de escape
    }
}

```

```

        System.out.println("\n=====");
        System.out.println("ENTIDAD:\t" + NOMBRE_INSTITUTO);
        System.out.println("SISTEMA:\t\"Control de Acceso Dinámico QR\"");
        System.out.println("UBICACIÓN:\tVestíbulo Principal \\ Edificio A");
        System.out.println("=====");
        System.out.println("FICHAJE N.º:\t#" + contadorFichajesTerminal + "\t\tESTADO:\tVALIDADO");
        System.out.println("ESTUDIANTE:\t" + nombreEstudiante);
        System.out.println("IDENTIFICADOR:\t" + dniEstudiante + "\tGRUPO:\t" + letraGrupo + " DAM");
        System.out.println("TOKEN QR:\t" + tokenResumen);
        System.out.println("-----");
        System.out.println("PERMANENCIA:\t" + minutosTotalesLectivos + " minutos lectivos.");
        System.out.println("OCUPACIÓN:\t" + porcentajeOcupacionReal + " % (Panel: " + porcentajeOcupacionTruncado + " %)");
        System.out.println("SERVICIO:\t" + horasUptime + " h " + minutosUptime + " m " + segundosUptime + " s");
        System.out.println("REGISTRO LOG:\t" + RUTA_LOGS);
        System.out.println("=====");

        teclado.close();
    }
}

```

4. Trabajo del estudiante: evolución de su proyecto propio

El estudiante abre su archivo `.java` (en v0.7):

- Sustituye las concatenaciones desalineadas por tabuladores `\t` para alinear las etiquetas de los datos.
- Añade comillas escapadas `\` para citar el nombre comercial de su cliente y barras `\\` para rutas de auditoría.
- Comprueba que la salida luce limpia y profesional.

2.5 ---

2.5.1 Día 11 - 2 sesiones

1. Caso guía en AzaharTech

Es miércoles por la tarde. **Laia Claramunt** revisa la versión v0.8:

«La tabulación con `\t` es un avance, pero tiene un punto débil: si el nombre de un alumno es muy largo (como 'Constantinopla'), el tabulador salta a la siguiente parada y rompe la columna. Además, los decimales del porcentaje siguen mostrando hasta quince dígitos.

Hoy alcanzaremos la versión v0.9: eliminaremos los `println` del ticket y utilizaremos `System.out.printf()`. Definiremos anchos de campo fijos, alinearemos textos a la izquierda y números a la derecha, y redondearemos los decimales automáticamente a exactamente dos posiciones».

2. Fundamento teórico: especificadores de formato en `System.out.printf()`

PLANTILLA DE FORMATO PROFESIONAL PRINTF		
Patrón	Función técnica	Ejemplo
<code>%-25s</code>	Texto alineado a la IZQUIERDA en un campo de 25 caracteres.	<code>%-25s -> "Juan Pérez"</code>
<code>%04d</code>	Entero rellenado con ceros a la izquierda hasta 4 dígitos.	<code>%04d -> "0007"</code>
<code>%6.2f</code>	Decimal con 2 decimales fijos en un ancho total de 6 caracteres.	<code>%6.2f -> " 98.75"</code>
<code>%n</code>	Salto de línea independiente	<code>%n</code>

3. Evolución a ControlAccesoQR v0.9

Actualizamos el bloque final de impresión de `ControlAccesoQR` sustituyendo los textos concatenados por una plantilla con `printf`.

PASO A. EVOLUCIÓN EN PSEINT (pr/pseudocodigo/ControlAccesoQR.psc — V0.9)

```

Algoritmo ControlAccesoQR
// =====
// SISTEMA DE CONTROL DE ASISTENCIA QR - IES EL CAMINAS (Castellon)
// Version: 0.9 (Evolucion: Formato Tabular de Precision)
// =====

// Constantes
Definir NOMBRE_INSTITUTO, PREFIJO_CENTRO Como Cadena
Definir MINUTOS_POR_SESION, SEGUNDOS_POR_HORA, SEGUNDOS_POR_MINUTO Como Entero
Definir FACTOR_PORCENTAJE Como Real

NOMBRE_INSTITUTO <- "IES El Caminas (Castellon)"
PREFIJO_CENTRO <- "CAMINAS"
MINUTOS_POR_SESION <- 50
SEGUNDOS_POR_HORA <- 3600
SEGUNDOS_POR_MINUTO <- 60
FACTOR_PORCENTAJE <- 100.0

// Variables
Definir terminalId, contadorFichajesTerminal, aforoMaximoVestibulo Como Entero
Definir tempVestibulo, porcentajeOcupacionReal Como Real
Definir porcentajeOcupacionTruncado Como Entero
Definir nombreEstudiante, dniEstudiante, tokenResumen Como Cadena
Definir letraGrupo Como Caracter
Definir matriculaActiva Como Logico
Definir sesionesManana, sesionesTarde, totalSesiones, minutosTotalesLectivos Como Entero
Definir minutosExtraGuardia, segundosActividadTerminal Como Entero
Definir horasUptime, minutosUptime, segundosUptime Como Entero

contadorFichajesTerminal <- 0

// Entrada de datos
Escribir "=== AZAHARTECH: TERMINAL IES EL CAMINAS (v0.9) ==="
Escribir "ID Terminal, temperatura y segundos de actividad:"
Leer terminalId
Leer tempVestibulo
Leer segundosActividadTerminal
Escribir "Aforo maximo del vestibulo:"
Leer aforoMaximoVestibulo

Escribir "DNI, nombre y grupo del alumno:"
Leer dniEstudiante
Leer nombreEstudiante
Leer letraGrupo
matriculaActiva <- Verdadero

Escribir "Sesiones de manana, tarde y minutos de guardia:"
Leer sesionesManana
Leer sesionesTarde
Leer minutosExtraGuardia

// Cálculos
contadorFichajesTerminal <- contadorFichajesTerminal + 1
totalSesiones <- sesionesManana + sesionesTarde
minutosTotalesLectivos <- (totalSesiones * MINUTOS_POR_SESION) + minutosExtraGuardia

horasUptime <- trunc(segundosActividadTerminal / SEGUNDOS_POR_HORA)
minutosUptime <- trunc((segundosActividadTerminal MOD SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO)
segundosUptime <- segundosActividadTerminal MOD SEGUNDOS_POR_MINUTO

porcentajeOcupacionReal <- (contadorFichajesTerminal * FACTOR_PORCENTAJE) / aforoMaximoVestibulo
porcentajeOcupacionTruncado <- trunc(porcentajeOcupacionReal)

tokenResumen <- PREFIJO_CENTRO + "-" + dniEstudiante + "-T" + ConvertirATexto(terminalId)

// Salida simulando redondeo a 2 decimales
Escribir ""
Escribir "=====
Escribir "          INFORME OFICIAL DE ACCESO EN VESTIBULO
Escribir "=====
Escribir "REGISTRO:  #", contadorFichajesTerminal, " | TOKEN: ", tokenResumen
Escribir "ESTUDIANTE:  ", nombreEstudiante, " | GRUPO: ", letraGrupo, " DAM"
Escribir "DNI:       ", dniEstudiante
Escribir "PERMANENCIA: ", minutosTotalesLectivos, " min lectivos."
Escribir "OCUPACION:  ", redon(porcentajeOcupacionReal * 100) / 100, " %"
Escribir "UPTIME:     ", horasUptime, "h ", minutosUptime, "m ", segundosUptime, "s"
Escribir "=====
FinAlgoritmo

```

PASO B. EVOLUCIÓN EN JAVA (pr/src/ControlAccesoQR.java — V0.9)

```

/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)

```

```

* Consultora: AzaharTech Software Consulting
*
* Versión 0.9: Salida formateada profesional con System.out.printf().
* Módulo: Programación (PR) - Sprint 1 (RA1)
*/

import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        // Constantes inmutables
        final String NOMBRE_INSTITUTO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final int MINUTOS_POR_SESION = 50;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;

        Scanner teclado = new Scanner(System.in);

        int terminalId;
        double tempVestibulo;
        int contadorFichajesTerminal = 0;
        int segundosActividadTerminal;
        int aforoMaximoVestibulo;

        String dniEstudiante;
        String nombreEstudiante;
        char letraGrupo;
        boolean matriculaActiva = true;

        int sesionesManana;
        int sesionesTarde;
        int totalSesiones;
        int minutosExtraGuardia;
        int minutosTotalesLectivos;

        int horasUptime;
        int minutosUptime;
        int segundosUptime;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionTruncado;
        String tokenResumen;

        // Entrada de datos
        System.out.println("=====");
        System.out.println("  AZAHARTECH - TERMINAL  " + NOMBRE_INSTITUTO);
        System.out.println("=====");
        System.out.print("ID Terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura sensor (°C): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos acumulados de actividad: ");
        segundosActividadTerminal = teclado.nextInt();

        System.out.print("Aforo máximo permitido en vestíbulo: ");
        aforoMaximoVestibulo = teclado.nextInt();

        teclado.nextLine(); // Limpieza obligatoria del buffer

        System.out.print("DNI Estudiante: ");
        dniEstudiante = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombreEstudiante = teclado.nextLine();

        System.out.print("Grupo (letra): ");
        letraGrupo = teclado.next().charAt(0);

        System.out.print("Sesiones turno mañana: ");
        sesionesManana = teclado.nextInt();

        System.out.print("Sesiones turno tarde: ");
        sesionesTarde = teclado.nextInt();

        System.out.print("Minutos de guardia/tutoría: ");
        minutosExtraGuardia = teclado.nextInt();

        // Procesamiento
        contadorFichajesTerminal++;
        totalSesiones = sesionesManana + sesionesTarde;
        minutosTotalesLectivos = (totalSesiones * MINUTOS_POR_SESION) + minutosExtraGuardia;

        horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
        minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
        segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

        porcentajeOcupacionReal = ((double) contadorFichajesTerminal / aforoMaximoVestibulo) * FACTOR_PORCENTAJE;
        porcentajeOcupacionTruncado = (int) porcentajeOcupacionReal;

        tokenResumen = PREFIJO_CENTRO + "-" + dniEstudiante + "-T" + terminalId;
    }
}

```

```
// [NUEVO DÍA 11] Salida formateada profesional con printf
System.out.println("\n=====");
System.out.println("                INFORME OFICIAL DE ACCESO EN VESTÍBULO                ");
System.out.println("=====");
System.out.printf("CENTRO:          %-30s\n", NOMBRE_INSTITUTO);
System.out.printf("REGISTRO N.º:      #05d | TOKEN: %s\n", contadorFichajesTerminal, tokenResumen);
System.out.printf("ESTUDIANTE:       %-30s | GRUPO: %c DAM\n", nombreEstudiante, letraGrupo);
System.out.printf("IDENTIFICADOR:    %-12s | MATRÍCULA ACTIVA: %b\n", dniEstudiante, matriculaActiva);
System.out.println("-----");
System.out.printf("PERMANENCIA:      %03d minutos lectivos (%d sesiones)\n", minutosTotalesLectivos, totalSesiones);
System.out.printf("AFORO OCUPADO:    %6.2f %% (Indicador panel: %03d %%)\n", porcentajeOcupacionReal, porcentajeOcupacionTruncado);
System.out.printf("SERVICIO ACTIVO:  %02dh %02dm %02ds (Sensor: %.1f °C)\n", horasUptime, minutosUptime, segundosUptime, tempVestibulo);
System.out.println("=====");

teclado.close();
}
}
```

4. Trabajo del estudiante: evolución de su proyecto propio

El estudiante abre su archivo `.java` (versión v0.8):

- Sustituye todas las instrucciones de salida por llamadas a `System.out.printf()`.
- Aplica especificadores con ancho fijo, enteros con ceros a la izquierda y redondeo a dos decimales.
- Ejecuta pruebas con distintos nombres para certificar que las columnas permanecen perfectamente alineadas.

2.6 ---

2.6.1 Día 12 - 2 sesiones

1. Caso guía en AzaharTech

Es jueves 1 de octubre. Mañana viernes concluye formalmente el **Sprint 1**. En la sala de juntas de **AzaharTech**, **Laia Claramunt** convoca a todo el equipo de desarrollo frente al proyector:

«Equipo, contemplad lo que hemos construido en doce días de trabajo: tenemos un software vivo, robusto y profesional que ha crecido día a día.

Hoy alcanzamos la versión v1.0: añadiremos la cabecera formal de documentación Javadoc, limpiaremos cualquier advertencia del compilador y dejaremos sellado el código de vuestro proyecto propio en GitHub para la evaluación de mañana».

2. Fundamento teórico: Documentación técnica y autoformato

- Comentarios Javadoc** (`/** ... */`). Permiten a las herramientas de ingeniería extraer manuales técnicos automáticos en formato HTML:
 - `@author`. Desarrollador o equipo de trabajo responsable.
 - `@version`. Número de versión semántica del software.
- Autoformato en IntelliJ** (`Ctrl + Alt + L`). Reorganiza el código para que respete las directrices internacionales de indentación (4 espacios) y separación de operadores.

3. El código definitivo del Sprint 1: `ControlAccesoQR v1.0`

Presentamos la versión íntegra y definitiva que el docente modela como cierre del caso guía:

```
/**
 * SISTEMA DE CONTROL DE ASISTENCIA POR CÓDIGO QR
 * Cliente: IES El Caminàs (Castellón de la Plana)
 * Consultora: AzaharTech Software Consulting
 *
 * Versión 1.0 (Definitiva Sprint 1):
```

```

* Programa secuencial integral que captura parámetros de terminal y usuario,
* realiza descomposiciones temporales y cálculos porcentuales de aforo sin pérdida
* de precisión decimal, y emite un informe oficial formateado mediante printf.
*
* @author Equipo AzaharTech (Alba Torres, Pau Ferrer)
* @version 1.0 (Octubre 2026)
* @since JDK 21 LTS
*/

import java.util.Scanner;

public class ControlAccesoQR {
    public static void main(String[] args) {
        // -----
        // 1. CONSTANTES INMUTABLES DEL SISTEMA (Configuración corporativa)
        // -----
        final String NOMBRE_INSTITUTO = "IES El Caminàs (Castellón)";
        final String PREFIJO_CENTRO = "CAMINAS";
        final int MINUTOS_POR_SESION = 50;
        final int SEGUNDOS_POR_HORA = 3600;
        final int SEGUNDOS_POR_MINUTO = 60;
        final double FACTOR_PORCENTAJE = 100.0;

        // -----
        // 2. DECLARACIÓN DE VARIABLES DE MEMORIA
        // -----
        Scanner teclado = new Scanner(System.in);

        // Variables de hardware y terminal
        int terminalId;
        double tempVestibulo;
        int contadorFichajesTerminal = 0; // Contador acumulativo
        int segundosActividadTerminal;
        int aforoMaximoVestibulo;

        // Variables de identidad y estado del estudiante
        String dniEstudiante;
        String nombreEstudiante;
        char letraGrupo;
        boolean matriculaActiva = true;

        // Variables de cómputo de sesiones
        int sesionesManana;
        int sesionesTarde;
        int totalSesiones;
        int minutosExtraGuardia;
        int minutosTotalesLectivos;

        // Variables de magnitudes descompuestas y ratios
        int horasUptime;
        int minutosUptime;
        int segundosUptime;
        double porcentajeOcupacionReal;
        int porcentajeOcupacionTruncado;
        String tokenResumen;

        // -----
        // 3. CAPTURA INTERACTIVA DE DATOS
        // -----
        System.out.println("=====");
        System.out.println("  AZAHARTECH - TERMINAL " + NOMBRE_INSTITUTO);
        System.out.println("  Versión 1.0 Oficial (Programa Secuencial Base)");
        System.out.println("=====");
        System.out.print("ID Terminal: ");
        terminalId = teclado.nextInt();

        System.out.print("Temperatura sensor (°C): ");
        tempVestibulo = teclado.nextDouble();

        System.out.print("Segundos acumulados de actividad: ");
        segundosActividadTerminal = teclado.nextInt();

        System.out.print("Aforo máximo permitido en vestíbulo: ");
        aforoMaximoVestibulo = teclado.nextInt();

        teclado.nextLine(); // Limpieza obligatoria del buffer de teclado

        System.out.print("DNI Estudiante: ");
        dniEstudiante = teclado.nextLine();

        System.out.print("Nombre completo: ");
        nombreEstudiante = teclado.nextLine();

        System.out.print("Grupo (letra): ");
        letraGrupo = teclado.next().charAt(0);

        System.out.print("Sesiones turno mañana: ");
        sesionesManana = teclado.nextInt();

        System.out.print("Sesiones turno tarde: ");
        sesionesTarde = teclado.nextInt();

        System.out.print("Minutos de guardia/tutoría: ");
        minutosExtraGuardia = teclado.nextInt();
    }
}

```

```
// -----
// 4. PROCESAMIENTO SECUENCIAL Y OPERACIONES MATEMÁTICAS
// -----
// Incremento unario del contador de accesos
contadorFichajesTerminal++;

// Suma y cálculo de permanencia lectiva
totalSesiones = sesionesManana + sesionesTarde;
minutosTotalesLectivos = (totalSesiones * MINUTOS_POR_SESION) + minutosExtraGuardia;

// Descomposición temporal exacta mediante división entera y módulo (%)
horasUptime = segundosActividadTerminal / SEGUNDOS_POR_HORA;
minutosUptime = (segundosActividadTerminal % SEGUNDOS_POR_HORA) / SEGUNDOS_POR_MINUTO;
segundosUptime = segundosActividadTerminal % SEGUNDOS_POR_MINUTO;

// Casting explícito a (double) para evitar la división entera a cero
porcentajeOcupacionReal = ((double) contadorFichajesTerminal / aforoMaximoVestibulo) * FACTOR_PORCENTAJE;
porcentajeOcupacionTruncado = (int) porcentajeOcupacionReal; // Casting a entero

// Construcción del token identificador QR
tokenResumen = PREFIJO_CENTRO + "-" + dniEstudiante + "-T" + terminalId;

// -----
// 5. SALIDA FORMATEADA PROFESIONAL (System.out.printf)
// -----
System.out.println("\n=====");
System.out.println("                INFORME OFICIAL DE ACCESO EN VESTÍBULO                ");
System.out.println("=====");
System.out.printf("CENTRO:           %-30s\n", NOMBRE_INSTITUTO);
System.out.printf("REGISTRO N.º:      #%05d | TOKEN: %s\n", contadorFichajesTerminal, tokenResumen);
System.out.printf("ESTUDIANTE:       %-30s | GRUPO: %c DAM%\n", nombreEstudiante, letraGrupo);
System.out.printf("IDENTIFICADOR:    %-12s | MATRÍCULA ACTIVA: %b\n", dniEstudiante, matriculaActiva);
System.out.println("-----");
System.out.printf("PERMANENCIA:      %03d minutos lectivos (%d sesiones)\n", minutosTotalesLectivos, totalSesiones);
System.out.printf("AFORO OCUPADO:    %6.2f %% (Indicador panel: %03d %%)\n", porcentajeOcupacionReal, porcentajeOcupacionTruncado);
System.out.printf("SERVICIO ACTIVO:  %02dh %02dm %02ds (Sensor: %.1f °C)\n", horasUptime, minutosUptime, segundosUptime, tempVestibulo);
System.out.println("=====");

// Cierre preventivo del recurso de entrada
teclado.close();
}
}
```

4. Trabajo del estudiante: la versión v1.0 de su proyecto propio

Cada estudiante finaliza su archivo `pr/src/MiProyecto.java`:

1. Incorpora la cabecera Javadoc con sus datos.
2. Aplica el formateador de código en IntelliJ (Ctrl + Alt + L).
3. Comprueba que el programa compila limpiamente y que la salida por consola con `printf` es una tabla perfectamente alineada.
4. Realiza el commit final del módulo de Programación para el Sprint 1:

```
git add pr/
git commit -m "feat(pr): consolidar version 1.0 del programa secuencial completo para sprint 1"
git push
```

2.6.2 Cierre del Sprint 1

- **Entrega lista.** El estudiante tiene su proyecto propio en GitHub en versión `v1.0`, listo para ser sellado mañana viernes bajo la etiqueta `v0.1.0-sprint1`.

3. Unidad 2

Aquí irá el contenido de la **Unidad 2**.